# Back-of-the-Envelope Estimation

*which numbers to ask for first*

Watch the 3-min video

**In one line:** before you estimate, ask for daily and monthly users, actions per user (reads and writes), object size, retention, peak vs average traffic and where users are. If a number is missing, state a round assumption out loud.

## 1 Why estimate first

"Design a URL shortener." Before you draw a single box you need to know: is this a hobby project with a **thousand links**, or a global service creating **a hundred million links a day**? The architecture is completely different. One gets a single small server and a database; the other needs caching, replication and sharding.

So the first step of any estimate is **asking for the right numbers**. The goal is not a precise answer. It is the right **order of magnitude**: 10, 1,000 or 100,000 requests per second, gigabytes or petabytes. That is what decides the design.

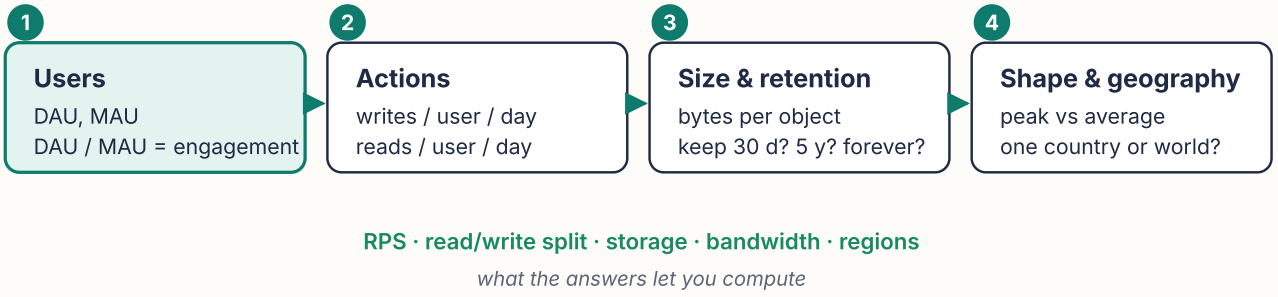



**Figure 1.** Four groups of questions, asked in this order. Each answer feeds a number you will need later.

## 2 Users: DAU and MAU

**DAU** means daily active users: people who use the product on a given day. **MAU** means monthly active users. Interviewers often give you one of them.

The ratio **DAU / MAU** tells you how engaged users are. A social app might see half of its monthly users every day (50%). A tool people open once a week sits far lower. If you know MAU and the ratio, you get DAU, and DAU drives traffic.

## 3 Actions, sizes and retention

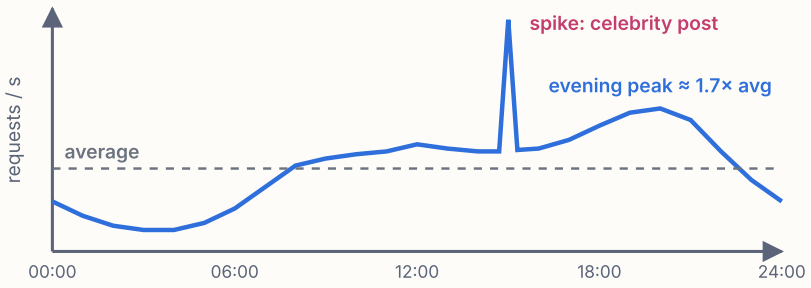
**Actions per user per day.** How many times a day does a user **write** something (post, upload, create a link)? How many times do they **read** (view a feed, open a link)? From that you get the **read-to-write ratio**. Most systems are read-heavy, often **10:1**, sometimes **100:1**. The ratio tells you whether to invest in caches and replicas or in write throughput.

**Size of each object.** A few hundred bytes for a database row, a few megabytes for a photo, gigabytes for a video. **Retention:** 30 days, 5 years, or forever?

```
storage = size per object × objects per day × days retained
```

## 4 Shape and geography

Is traffic flat, or does it spike: at lunchtime, on Black Friday, or when a celebrity posts? **Peak is often 2–10× the average**, and you size for the peak. And where are the users: one country, or worldwide? That decides how many regions you run, how you replicate data, and whether you need a CDN.




**Ask about shape**
is it flat, or does it spike?
peak ≈ 2–10× average

**Ask about geography**
one country or worldwide?
→ regions, replication, CDN

**Figure 2.** An average hides the evening peak and the sudden spike. Ask for the shape, not just the daily total.

## 5 Worked example: the URL shortener

Ask the questions, get the answers, then compute. Use 86,400 seconds per day (about $10^5$ for quick math).

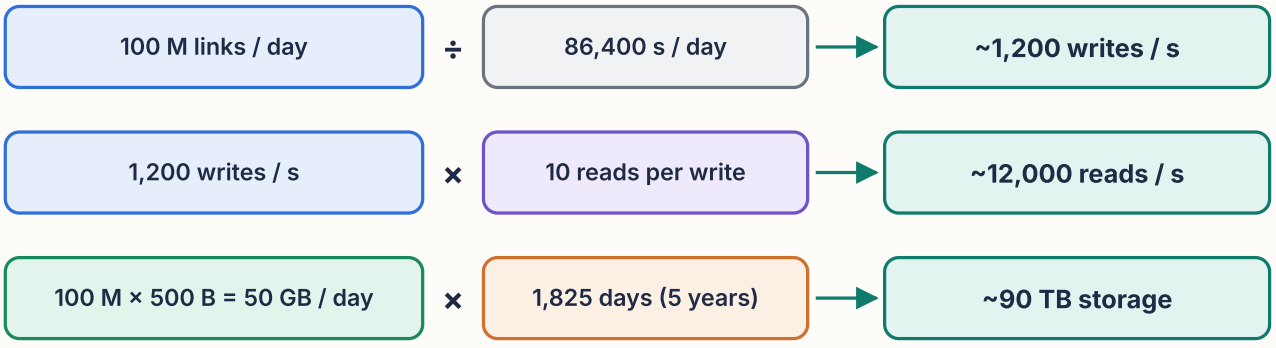



**Figure 3.** Four answers give three numbers: write rate, read rate and storage.

**WORKED EXAMPLE**

- **Writes:** 100 M links ÷ 86,400 s ≈ **1,157**, call it **~1,200 writes/s**.
- **Reads:** 10 redirects per link → 10 × 1,157 ≈ **~12,000 reads/s**. Read-heavy: cache the hot short codes.
- **Storage:** 100 M × 500 B = 50 GB/day; × 1,825 days ≈ **91 TB**, about **90 TB** before replication. That is about 183 billion links: too much for one comfortable database node, so plan to shard.

## 6 When the interviewer gives no number

Don't freeze. **State an assumption out loud**: “Let's assume 100 million links a day.” Round it so the math is easy, and **write it down** where both of you can see it. The interviewer can correct you, and your reasoning stays easy to follow. A wrong but stated assumption is fine; a silent one is not.

**COMMON MISTAKES**

- Drawing the architecture before knowing whether it is 10 or 10,000 requests per second.
- Chasing exact arithmetic (1,157.4 writes/s) instead of the order of magnitude (~1,000).
- Forgetting retention, so storage is computed for one day instead of five years.
- Sizing for the daily average and ignoring the 2–10× peak.
- Making assumptions silently, so the interviewer can't follow or correct them.

## 7 Key terms

**DAU / MAU**: daily / monthly active users

**Engagement**: DAU ÷ MAU, e.g. 50%

**Read:write ratio**: reads per write, often 10:1 to 100:1

**Retention**: how long data must be kept

**Peak factor**: peak ÷ average traffic, often 2–10×

**Order of magnitude**: the power of ten: $10^3$ vs $10^5$

**Back-of-the-envelope**: quick, rounded estimate to pick a design

**Assumption**: a stated, rounded number used when none is given

**RPS**: requests per second

## 8 Interview questions

**Q1. Which numbers do you ask for before estimating?** BASIC

DAU and MAU, actions per user per day split into reads and writes, the read-to-write ratio, the size of each object, retention, peak vs average traffic, and where users are located.

**Q2. What does the DAU/MAU ratio tell you?** BASIC

Engagement: the share of monthly users active on a given day. 100 M DAU out of 200 M MAU is 50%, typical of a sticky social app; a weekly tool might be closer to 15–20%.

**Q3. How do you compute storage?** BASIC

Size per object × objects per day × days retained. 100 M records/day × 500 B × 1,825 days ≈ 91 TB, before replication (× 3 copies ≈ 270 TB of disk).

**Q4. Estimate the URL shortener: 100 M links/day, 10:1 reads, 5 years, 500 B each.** SCENARIO

~1,200 writes/s, ~12,000 reads/s and ~90 TB over five years. Read-heavy, so cache redirects; 90 TB means the link table will be sharded.

**Q5. The interviewer won't give you a DAU. What do you do?** SCENARIO

State a round assumption out loud ("let's assume 10 million DAU"), write it down, and carry on. Offer to redo the math if they prefer another number; the method matters more than the input.

**Q6. Why ask where users are located?** SCENARIO

It decides regions, replication and CDN. Worldwide users far from one region see 150 ms+ round trips, so you may serve reads from several regions and static content from a CDN.

**Q7. Your estimate says 1,157 writes per second. How do you present it?** TRICKY

As ~1,000 (or ~1,200) writes per second. Estimates are for orders of magnitude; false precision hides the fact that the inputs were rough guesses.

**Q8. Why does retention matter more than daily volume for storage?** TRICKY

Storage grows with every day you keep. 50 GB/day is trivial for one day but ~91 TB over five years; with "forever" it never stops growing, so you also need archiving or tiering.

Next: SD-D-002 · Latency Numbers & Powers of Two backendroadmap.com

# Latency Numbers & Powers of Two

*the handful of numbers to know by heart*

Watch the 3-min video

**In one line:** know powers of two (KB, MB, GB, TB, PB), seconds per day (86,400 ≈ $10^5$), and the latency ladder from 1 ns (L1) to 150 ms (across continents). With them you turn "1 million requests a day" into "about 12 per second" in two seconds.

## 1 Why memorise a few numbers

An interviewer says: "we get one million requests a day. Is that a lot?" Most people reach for a calculator. An experienced engineer answers in two seconds: **about twelve requests per second**. A single server handles that easily. The trick is a handful of numbers you know by heart: powers of two, seconds in a day, and latency orders of magnitude.

## 2 Powers of two: data sizes

$2^{10}$ = 1,024, roughly a thousand. That is a **kilobyte**. $2^{20}$ is about a million, a **megabyte**. $2^{30}$ is about a billion, a **gigabyte**. Then a **terabyte** (about a trillion bytes) and a **petabyte** (about a thousand trillion). Each step is a thousand times bigger.

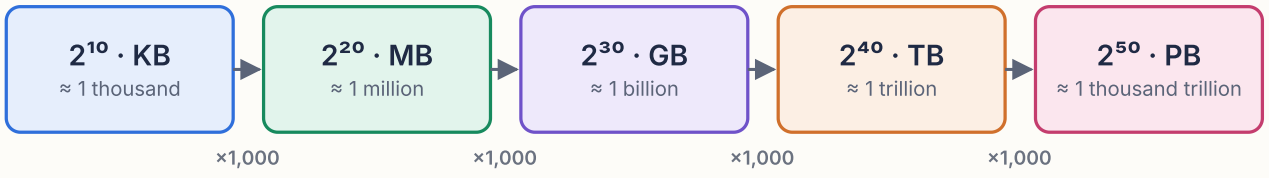



**Figure 1.** The powers-of-two ladder. For estimates, treat $2^{10}$ as 1,000; the 2.4% difference per step does not change any design decision.

## 3 Time: seconds in a day, month and year

A day has **86,400 seconds**. Round it to **100,000** ($10^5$). So one million per day is about 10 per second, and more precisely about 12. A month is about **2.5 million** seconds and a year about **30 million**.

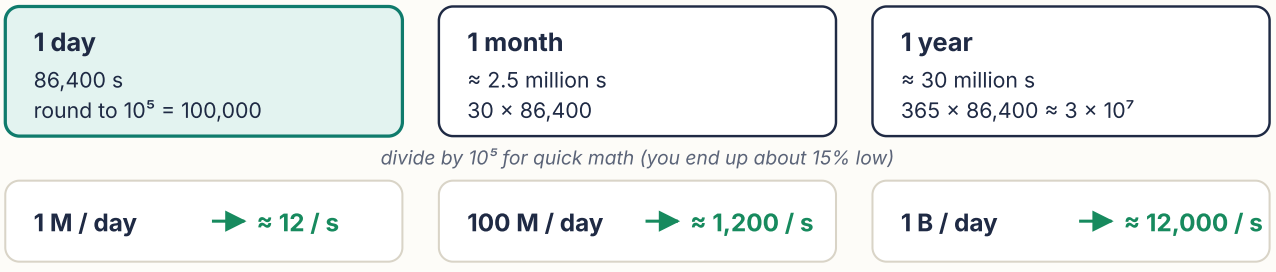



**Figure 2.** The time constants, and the three per-day → per-second conversions worth knowing cold.

## 4 The latency ladder

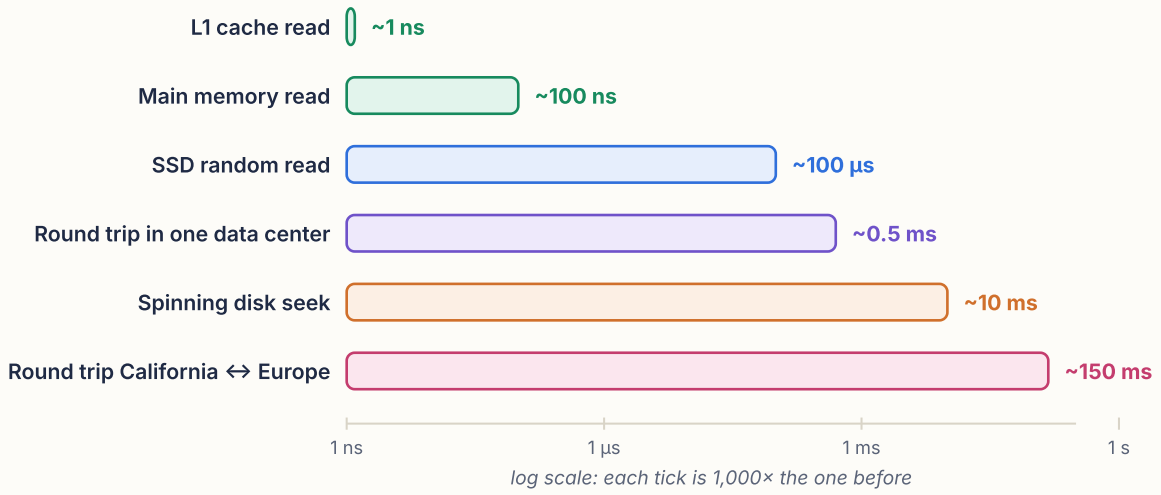



**Figure 3.** Orders of magnitude, not exact figures: hardware varies 2–3×, but the gaps between rungs stay huge.

| Operation | Latency | If a memory read took 1 second... |
|---|---|---|
| L1 cache read | ~1 ns | 0.01 s |
| Main memory read | ~100 ns | **1 s** |
| SSD random read | ~100 µs | ~17 minutes |
| Round trip in one data center | ~0.5 ms | ~1.4 hours |
| Spinning disk seek | ~10 ms | ~28 hours: more than a day |
| Round trip California ↔ Europe | ~150 ms | ~17 days |

**How much slower is a disk seek than a memory read?** 10 ms ÷ 100 ns = $10^{-2} \div 10^{-7}$ = **100,000×**. That is why caches, indexes and sequential reads matter so much.

## 5 How the numbers drive design

- **Memory is fast but limited**, so cache the hot data in RAM.
- **A network hop inside a data center (~0.5 ms) is cheap** compared with a disk seek (~10 ms). That is why a remote cache like Redis is still fast.
- **Cross-continent calls are slow (~150 ms)**, and no amount of hardware fixes the speed of light. Serve users from a nearby region, or a CDN.
- **Avoid many sequential calls in one request.** Their latencies add up: five calls across continents, one after another, cost about 750 ms before any work is done.

**WORKED EXAMPLE: 100 MILLION REQUESTS A DAY**

- **Average RPS:** 100 M ÷ $10^5$ s ≈ **1,000 requests/s** (exactly: ~1,157).
- **Peak:** a few thousand per second, at a 2–3× peak factor.
- **Bandwidth:** 1,000 × 10 KB = **10 MB/s** on average, about 80 Mbit/s. A single 10 Gbit/s network card has plenty of room.

**COMMON MISTAKES**

- Mixing up units: ms vs µs vs ns, or MB (bytes) vs Mb (bits, 8× smaller).
- Dividing by 86,400 with a calculator in the interview instead of using $10^5$.
- Treating a network call as "slow" by default: inside one data center it is faster than a disk seek.
- Chaining calls across regions in one request and expecting low latency.

## 6 Key terms

**Order of magnitude**: the power of ten: 10 ms vs 100 ms

**Round trip (RTT)**: time for a message to go and come back

**KB / MB / GB / TB / PB**: ≈ $10^3$ / $10^6$ / $10^9$ / $10^{12}$ / $10^{15}$ bytes

**Seek**: moving a disk head to the data

**ns / µs / ms**: $10^{-9}$ / $10^{-6}$ / $10^{-3}$ seconds

**Sequential read**: reading data stored next to each other

## 7 Interview questions

**Q1. How many seconds are in a day?** BASIC

86,400. Round to $10^5$ for quick math, so 1 million requests a day is about 10 per second, and more precisely about 12.

**Q2. How much slower is a disk seek than a memory read?** BASIC

About 10 ms versus 100 ns: roughly 100,000× slower. If a memory read took one second, a disk seek would take more than a day.

**Q3. What is $2^{30}$ bytes, roughly?** BASIC

About a billion bytes: a gigabyte. $2^{10}$ ≈ thousand (KB), $2^{20}$ ≈ million (MB), $2^{40}$ ≈ trillion (TB).

**Q4. 100 million requests a day, 10 KB each. Estimate RPS and bandwidth.** SCENARIO

100 M ÷ $10^5$ ≈ 1,000 RPS on average, a few thousand at peak. 1,000 × 10 KB = 10 MB/s (about 80 Mbit/s) on average; size for the peak.

**Q5. Your service in Virginia calls a database in Frankfurt four times per request. What is the problem?** SCENARIO

Each round trip across the Atlantic is roughly 80–100 ms, so four sequential calls cost 300–400 ms before any work. Move the data or service into the same region, batch the calls, or cache.

**Q6. Why is a Redis cache over the network still fast?** SCENARIO

A round trip inside one data center is about 0.5 ms and Redis serves from RAM. That is about 20× faster than one spinning-disk seek (~10 ms) and much faster than a database query that touches disk.

**Q7. Is a round trip inside a data center faster than an SSD random read?** TRICKY

No. An SSD random read is about 100 µs; a data-center round trip is about 500 µs, so the SSD is roughly 5× faster. Both are far faster than a spinning-disk seek or a cross-continent call.

**Q8. Why do sequential reads matter so much on a spinning disk?** TRICKY

Each seek costs ~10 ms, then data streams quickly. Reading 1 MB in one run costs one seek; reading it as 256 random 4 KB blocks costs 256 seeks, over 2.5 seconds. That is why databases favour logs and large pages.

Next: SD-D-003 · How to Estimate RPS backendroadmap.com

# How to Estimate RPS

*DAU, reads vs writes, fan-out and peak*

Watch the 3-min video

**In one line:** RPS = DAU × actions per user per day ÷ 86,400 (≈ $10^5$). Then split reads from writes, multiply by the calls each action makes, and apply a peak factor. Size for the peak.

## 1 The question behind the number

Your interviewer says: "a hundred million daily active users, and each one reads their feed ten times a day. How many requests per second must the backend handle?" If you can turn that sentence into a number in thirty seconds, you will know whether you need **one server or a thousand**.

## 2 The core formula

```
RPS = DAU × actions per user per day ÷ 86,400
```

A day has 86,400 seconds. For quick math, round it to **100,000**. Your answer comes out about 14% low, which is fine for an estimate: you only need the right order of magnitude.

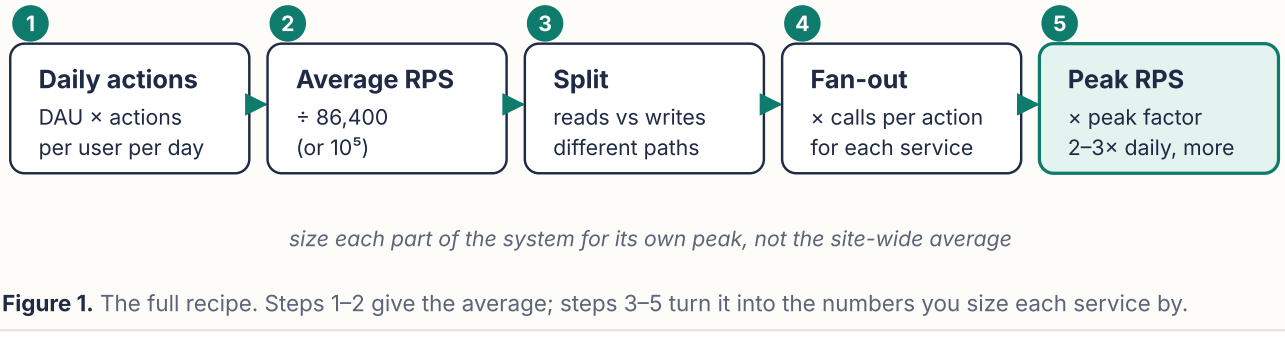



**Figure 1.** The full recipe. Steps 1–2 give the average; steps 3–5 turn it into the numbers you size each service by.

**WORKED EXAMPLE: THE FEED**

100 M users × 10 reads = **1 billion reads a day**. Divide by 86,400 → about **11,600 reads per second** on average. With the rounded shortcut, $10^9 \div 10^5$ = **10,000**. Either answer is the right order of magnitude.

## 3 Always split reads and writes

Reads and writes hit **different parts of the system**. Reads go to caches and read replicas, which scale out easily. Writes go to the primary database and to queues, which are harder to scale. So a single "total RPS" hides the number that matters for each part.

If those same 100 M users each post once a day, that is 100 M writes a day, about **1,200 writes per second**, against about **11,600 reads per second**.

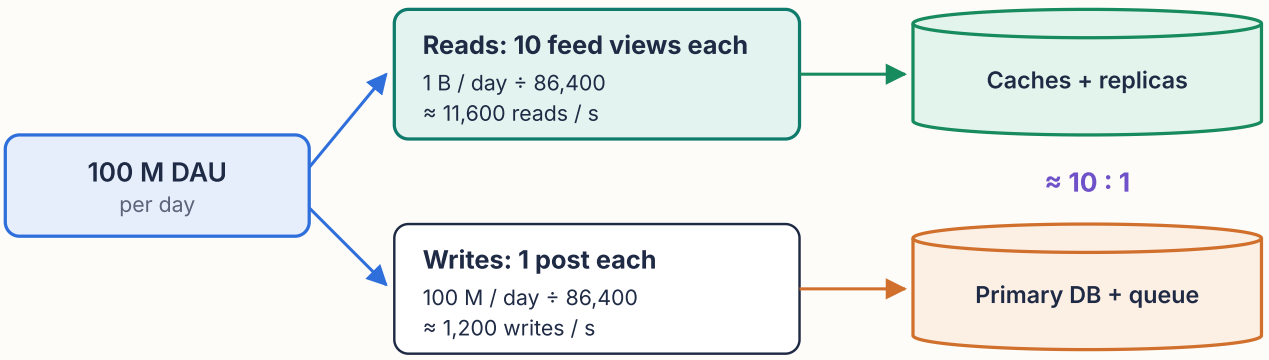



**Figure 2.** The same users produce two very different loads. Size the read path and the write path separately.

## 4 One action is rarely one request: fan-out

Opening a feed might call the feed service, the user service for avatars, the like counts and the ads service. That is a **fan-out** of four or more. So, for each service you size, multiply by the number of calls it receives per user action. **Internal traffic is often several times larger than external traffic.**

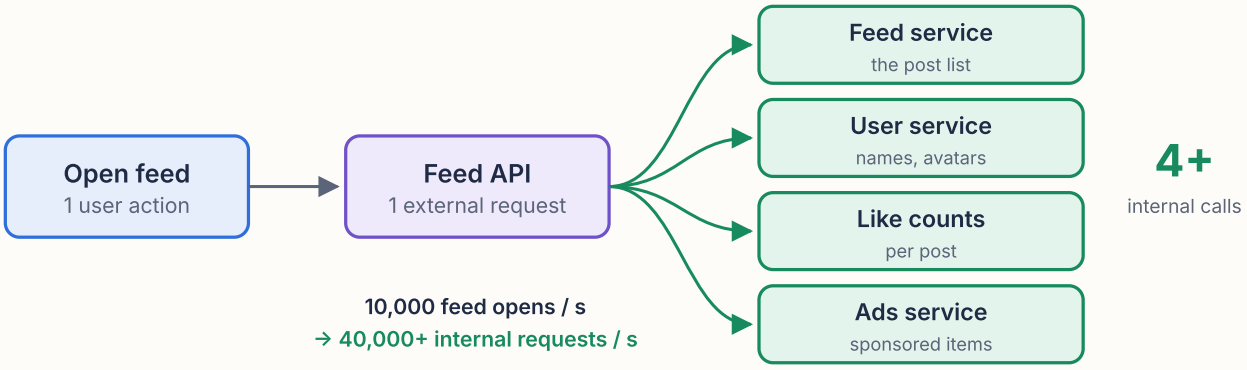



**Figure 3.** One tap becomes one external request and at least four internal ones. The user service sees every feed open, not just profile views.

## 5 Then apply the peak factor

Everything so far is the **average**. Real traffic follows the day, with evening peaks, and has sudden spikes. Multiply the average by a **peak factor**: often **2–3×** for daily patterns, and much more for events. Size the system for the peak, not the average. The next lesson covers this in detail.

**WORKED EXAMPLE: A CHAT APP**

- **Input:** 50 M DAU, 20 messages each per day → **1 billion messages a day**.
- **Average writes:** $10^9 \div 10^5 \approx 10{,}000$/s; more precisely $10^9 \div 86{,}400 \approx$ **11,600 messages/s**.
- **Reads:** each message is delivered to at least one recipient, so reads are at least as high: ≥ 11,600/s, and more for group chats.
- **Peak:** at a 3× peak factor, plan for about **35,000 writes/s**.

**COMMON MISTAKES**

- Giving one total RPS and never splitting reads from writes.
- Forgetting fan-out, so internal services are sized for a quarter of their real load.
- Sizing for the average and ignoring the peak.
- Spending a minute on exact long division when $10^5$ gets the order of magnitude in seconds.

## 6 Key terms

**RPS / QPS**: requests (queries) per second

**DAU**: daily active users

**Actions per user**: how often each user reads or writes per day

**Read/write split**: separate estimates for each path

**Fan-out**: internal calls made per user action

**Peak factor**: peak ÷ average, often 2–3× daily

## 7 Interview questions

**Q1. How do you estimate requests per second?** BASIC

DAU × actions per user per day ÷ 86,400 (or $10^5$ for quick math). Then split reads from writes, multiply by calls per action, and apply a peak factor.

**Q2. Why split reads from writes?** BASIC

They hit different parts of the system. Reads go to caches and replicas, writes to the primary database and queues. 11,600 reads/s and 1,200 writes/s call for very different designs.

**Q3. How wrong is it to divide by 100,000 instead of 86,400?** BASIC

About 14% low (86,400 ÷ 100,000 = 0.864). For an estimate that is fine: it never changes the order of magnitude.

**Q4. 50 M DAU, each sends 20 messages a day. Estimate RPS.** SCENARIO

1 B messages/day ÷ 86,400 ≈ 11,600/s on average (≈ 10,000 with $10^5$). Reads are at least as high. At a 3× peak factor, plan for about 35,000 writes/s.

**Q5. 100 M DAU open the feed 10 times a day, and each open calls 4 services. How much traffic does the user service get?** SCENARIO

Feed opens: 1 B/day ≈ 11,600/s on average. If every open calls the user service once, it sees ~11,600/s too; the four services together receive ~46,000 internal requests/s. At 3× peak, the user service needs ~35,000/s.

**Q6. An estimate gives 500 RPS at peak. What does that tell you?** SCENARIO

A couple of ordinary servers (plus one for redundancy) and a single database can handle it. You don't need sharding or a complex cache layer yet.

**Q7. Why is internal traffic often larger than external traffic?** TRICKY

Fan-out: one user action calls several services, and those may call others. A page that calls 4 services, each making 2 database queries, turns 1 external request into 12 internal operations.

**Q8. Your DAU is 10 M but MAU is 100 M. Which do you use for RPS?** TRICKY

DAU: it drives daily traffic. MAU matters for storage (accounts, profiles) and for growth. Using MAU would overestimate daily traffic about 10×.

Next: SD-D-004 · Peak vs Average Traffic backendroadmap.com

# Peak vs Average Traffic

*how to survive a 50× spike*

Watch the 3-min video

**In one line:** users feel the peak, not the average. Size for the expected peak plus 30–50% headroom, and absorb spikes with a CDN, a waiting room, queues, pre-scaling, rate limiting and graceful load shedding.

## 1 The 10 o'clock problem

A concert ticket sale opens at 10:00. On a normal day your site handles **2,000 requests per second**. At 9:59, a million fans are refreshing the page. At 10:00, traffic jumps to **100,000 requests per second**: fifty times normal. If you sized for the average, the site is already down.

## 2 Traffic is never flat

Most apps follow a **daily (diurnal) pattern**: quiet at night, rising in the morning, peaking in the evening. The busiest hour is often **2–3× the daily average**. Weekly and seasonal patterns stack on top: Monday mornings for business apps, Black Friday for shops.

Then there are **spikes**, or flash crowds: a ticket sale, a product launch, a celebrity post, a push notification sent to every user at once, breaking news. They can be **10–100× normal**, and they arrive in seconds, faster than autoscaling can react (starting new servers usually takes minutes).

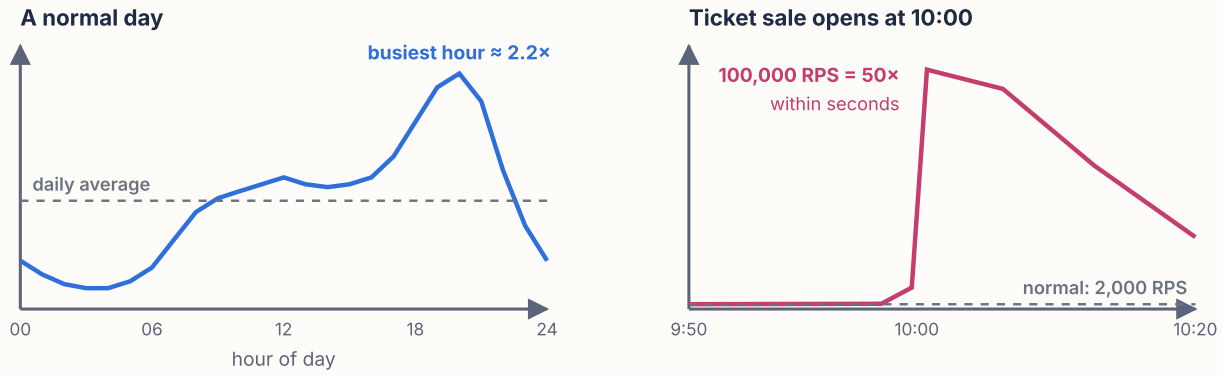



**Figure 1.** Left: the daily pattern; the busiest hour is about 2–3× the average. Right: a flash crowd; 50× in seconds, which no daily pattern predicts.

## 3 Why design for peak, not average

Because **users experience the peak**. Averages hide the moment when queues fill up, latency explodes and requests time out. A system at 100% utilization has no room to absorb a burst, and as it nears 100%, waiting time climbs steeply (see SD-A-009 on throughput). Once requests time out, clients retry, adding even more load: a **cascading failure**.

So size for the **expected peak plus headroom**, typically 30–50%. With 50% headroom, servers sit at about two-thirds busy at peak, in the 60–70% range most teams aim for.

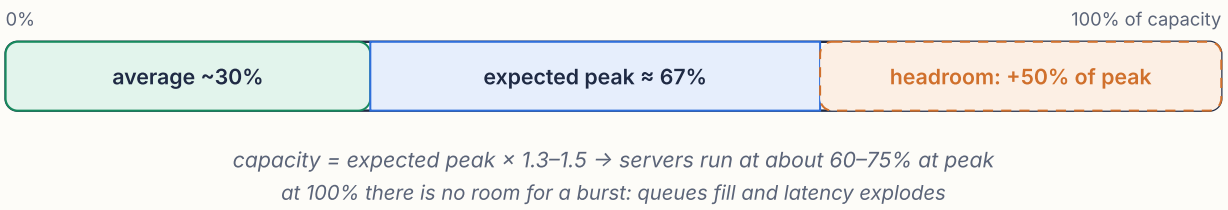



**Figure 2.** Average, peak and headroom on one capacity bar. The headroom is for the burst you did not predict and the server you just lost.

## 4 How to absorb a 50× spike

**First, don't do the slow work synchronously.** Put a **queue** between the front door and the slow part. Accept the request, enqueue it, reply “we're on it”, and process it at the rate the backend can sustain. A **virtual waiting room** does the same for users: let people in at a controlled rate, and show everyone else their place in line.

Then add more layers of defense:

- **CDN for static content.** Serve the event page from the edge, so origin servers never see most requests.
- **Pre-scale before a known event**, instead of waiting for autoscaling to notice the spike.
- **Rate limit per user** (429 Too Many Requests) so one client cannot refresh 50 times a second.
- **Shed load gracefully**: return “try again later” (503 with Retry-After) instead of crashing.
- **Turn off non-essential features** during the peak, such as recommendations or reviews.

## 5 Worked example: the ticket sale

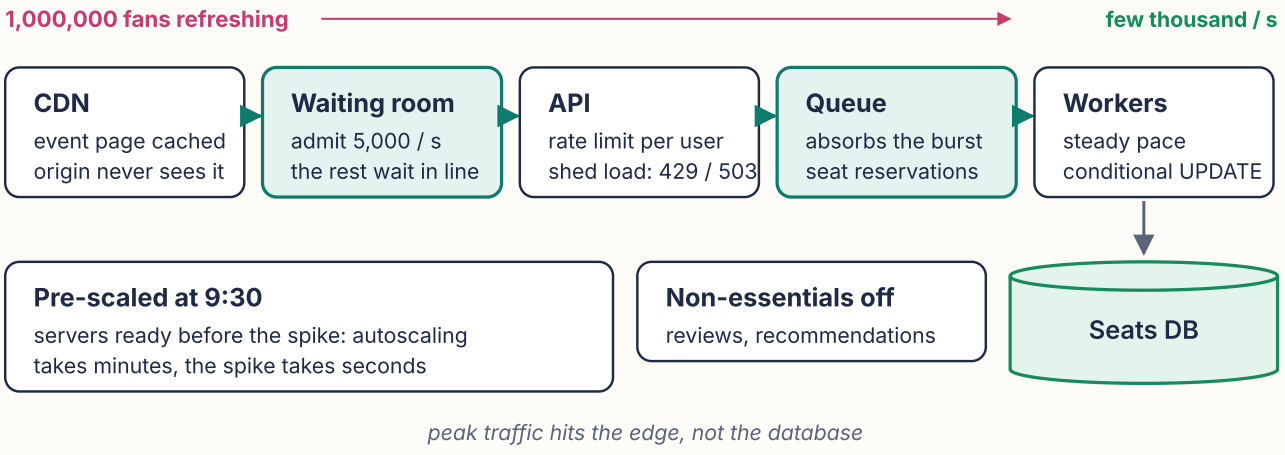



**Figure 3.** Each layer cuts the traffic that reaches the next. A million refreshing fans become a few thousand writes per second at the database.

**WORKED EXAMPLE**

- The **event page** is cached at the CDN, so the 100,000 RPS of page loads hit the edge.
- Fans enter a **waiting room** and are admitted at **5,000 per second**.
- Each **seat reservation goes into a queue**; workers process it with a row lock or a conditional update (`… WHERE seat_id = ? AND status = 'free'`), so nothing is oversold.
- The servers were **scaled up at 9:30**, before the sale opened.

Peak traffic hits the edge, not the database.

**COMMON MISTAKES**

- Sizing capacity from the daily average.
- Relying on autoscaling for a spike that arrives in seconds.
- Doing slow work (payments, seat allocation) inside the request instead of a queue.
- No rate limits or load shedding, so overload means a crash instead of “try again later”.
- Retrying immediately on timeout, which multiplies the spike.

## 6 Key terms

**Diurnal pattern**: daily rise and fall of traffic

**Peak factor**: peak ÷ average

**Flash crowd**: sudden spike, 10–100× normal

**Headroom**: spare capacity above the expected peak

**Virtual waiting room**: admits users at a controlled rate

**Load shedding**: rejecting some work to keep the rest healthy

**Pre-scaling**: adding capacity before a known event

**Cascading failure**: overload and retries spread from one part to all

**Rate limiting**: capping requests per user or key

## 7 Interview questions

**Q1. Why design for peak rather than average?** BASIC

Users feel the peak. A system sized for the average fails exactly when traffic is highest: queues fill, requests time out, retries add load and failures cascade. Size for the peak plus 30–50% headroom.

**Q2. What is a typical peak factor?** BASIC

For daily patterns, the busiest hour is often 2–3× the daily average. Flash crowds (launches, ticket sales, push notifications) can reach 10–100×.

**Q3. Why not run servers at 100% at peak?** BASIC

There is no room for a burst or a lost server, and latency rises steeply near full utilization. Teams aim for about 60–70% at peak.

**Q4. How would you absorb a 50× spike?** SCENARIO

CDN for static content, a waiting room and queues to smooth the burst, pre-scaling before known events, per-user rate limiting, graceful load shedding (503 + Retry-After), and turning off non-essential features.

**Q5. Average traffic is 2,000 RPS, the daily peak factor is 3, and you want 50% headroom. How much capacity?** SCENARIO

Peak = 2,000 × 3 = 6,000 RPS. With 50% headroom, 6,000 × 1.5 = 9,000 RPS. If one server handles 1,000 RPS, that is 9 servers, plus at least one spare for failures.

**Q6. A push notification to 20 M users causes a spike every time. What do you change?** SCENARIO

Send it in batches over a few minutes instead of all at once, add jitter on the client before it fetches, and serve the target content from the CDN or cache.

**Q7. Why can't autoscaling handle a ticket sale on its own?** TRICKY

It reacts to metrics that lag, and new servers take minutes to boot and warm up. A flash crowd arrives in seconds, so the site is overloaded before new capacity is ready. Pre-scale and use a waiting room.

**Q8. Isn't returning errors during a spike just failing?** TRICKY

Shedding some requests with a quick “try again later” keeps the system healthy for everyone already admitted. Without it, all requests slow down, time out and retry, and nobody gets served.

Next: SD-D-005 · Read/Write Ratio

# Read/Write Ratio: How 100:1 Reads Shape Your Architecture

*one ratio that changes almost every design decision*

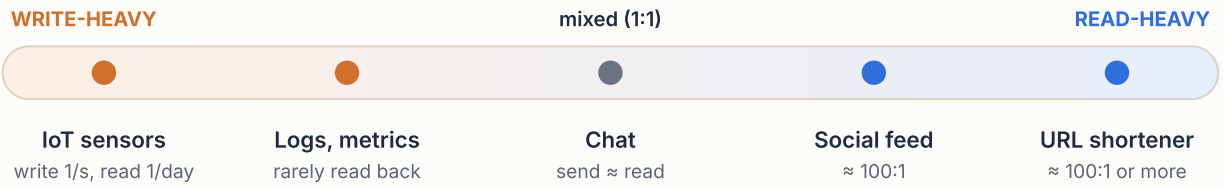
Watch the 3-min video

**In one line:** find the read-to-write ratio early; read-heavy systems (100:1) are built cache-first with replicas, CDNs and precomputed data, write-heavy systems use LSM stores, partitioning, batching and queues, and both must watch write amplification.

## 1 Two systems with opposite shapes

A short link is created once and then clicked a hundred times. A sensor reading is written once a second and read maybe once a day. Both store small records, yet they need almost opposite designs. The difference is one number: the **read-to-write ratio**.

The ratio compares how often data is read with how often it is written. **Read-heavy**: social feeds, product catalogs, URL shorteners, often 100:1 or more. **Write-heavy**: logging, metrics, IoT and chat ingestion. If the interviewer does not give it, ask. If they don't know, estimate it from actions per user: “each user posts 2 times a day and opens the feed 20 times, each feed load reads 50 posts” gives 2 writes vs 1,000 post reads, about 500:1.




*ask for the ratio early, or estimate it from actions per user*

**Figure 1.** Where common systems sit. The same app can sit in different places for different features.

## 2 Read-heavy: keep reads away from the primary

The goal is to serve reads **without touching the primary database**. Four tools, usually used together:

- **Cache first.** Put Redis or Memcached in front, so most reads are memory lookups (well under a millisecond) instead of database queries.
- **Read replicas** serve what the cache misses, so the primary mostly does writes.
- **CDN** for static content: images, scripts, video, even cacheable API responses.
- **Precompute.** Denormalized views and materialized feeds do the expensive joins once, at write time, so each read is cheap.

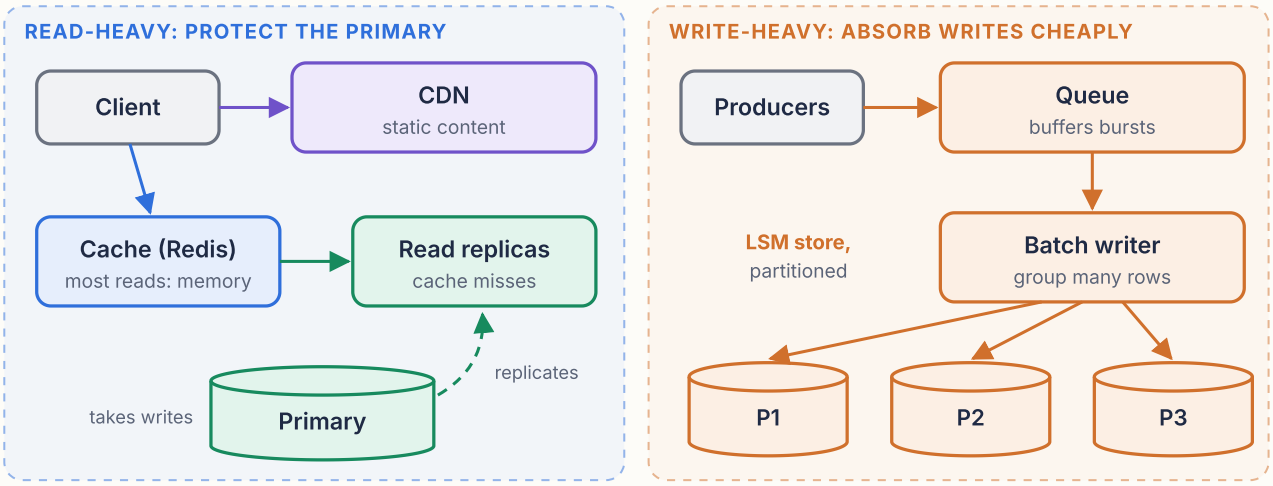



**Figure 2.** Left: reads hit the CDN or cache, then replicas; the primary takes the rare writes. Right: writes are queued, batched and spread over partitions of a write-optimized store.

## 3 Write-heavy: absorb writes cheaply

- **Write-optimized storage.** LSM-tree databases such as Cassandra (also RocksDB, ScyllaDB) append to a log and an in-memory table, then flush sorted files. They avoid updating pages in place, which is what B-tree databases do.
- **Partition the data** so writes spread across many nodes, not one primary. Pick a key that spreads load, such as sensor ID, not a timestamp.
- **Batch** writes together: one write of 500 rows beats 500 single-row writes.
- **Queue in front** (Kafka, SQS) to buffer bursts, so the store sees a smooth rate instead of spikes.

## 4 Watch out for write amplification

One **logical** write can become many **physical** writes:

- Every **secondary index** adds a write.
- **Replication** multiplies writes by the replication factor.
- **LSM compaction** rewrites the same data several times as files are merged.
- **Fan-out on write**, like pushing a post into a million followers' feeds, turns one write into a million.

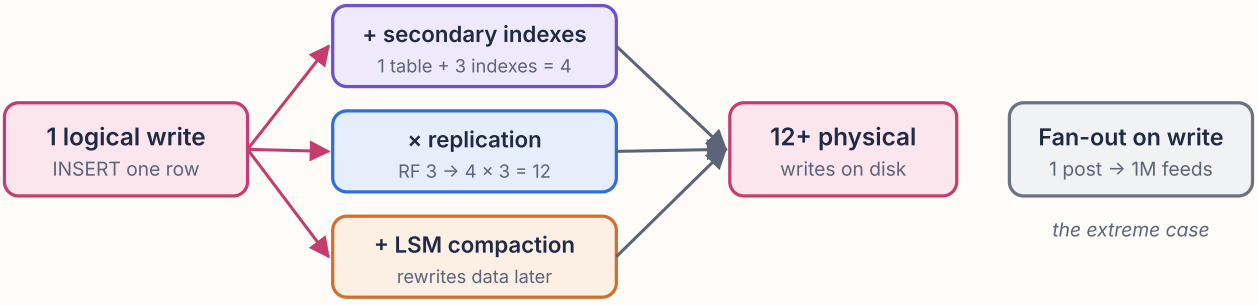



**Figure 3.** One insert with 3 secondary indexes and replication factor 3 is already 12 physical writes, before compaction.

So when you size a write-heavy system, size it for physical writes, not the number of API calls. Drop indexes you don't query, and think twice before fan-out on write for users with huge follower counts.

## 5 The URL shortener, and mixed systems

**WORKED EXAMPLE: URL SHORTENER (≈100:1)**

Say **100 million** new links a day: 100M ÷ 86,400 ≈ **1,200 writes/s**. At 100:1, that is about **120,000 redirects/s**.

Design **cache first**: on a redirect, look up the short code in Redis. Hot links stay in the cache, and the hit rate can reach **99%**. Only 1% misses, about 1,200 reads/s, go to the database: easy for a primary and a couple of replicas. Links never change once created, so **cache invalidation is trivial**. Writes are rare and simple: one insert per new link.

**Many systems are mixed, and the ratio differs per feature.** In a chat app, sending messages is write-heavy, but loading user profiles is read-heavy. So analyze the ratio **per data type** and pick the right tool for each: a cache for profiles, a write-optimized store for messages.

**COMMON MISTAKES**

- Never asking for the ratio, and designing one generic database for everything.
- Using one ratio for the whole app when features differ (messages vs profiles).
- Counting API writes but forgetting indexes, replication and fan-out.
- Adding read replicas first when a cache with a 99% hit rate would remove most of the load.

## 6 Key terms

**Read/write ratio**: reads per write, e.g. 100:1

**Read replica**: copy of the DB that serves reads

**Write amplification**: one logical write → many physical

**Read-heavy**: reads dominate; optimize the read path

**Denormalization**: store data pre-joined for cheap reads

**Fan-out on write**: copy a post into each follower's feed

**Write-heavy**: writes dominate; optimize ingestion

**LSM tree**: append-and-merge storage built for writes

**Hit rate**: share of reads served by the cache

## 7 Interview questions

**Q1. How does a 100:1 read ratio influence your design?** BASIC

It is read-heavy, so optimize the read path: cache first with a high hit rate, read replicas for misses, a CDN for static content and precomputed or denormalized data. Keep the rare, simple writes on the primary.

**Q2. How do you design for a write-heavy workload?** BASIC

Use an LSM-based store such as Cassandra, partition so writes spread over many nodes, batch writes, and put a queue in front to absorb bursts. Keep indexes minimal.

**Q3. What is write amplification?** BASIC

One logical write turning into many physical ones: indexes, replication, compaction, fan-out. One insert with 3 indexes at replication factor 3 is 12 physical writes.

**Q4. Nobody gives you the ratio for a news feed. How do you estimate it?** SCENARIO

From actions per user: say 2 posts a day and 20 feed loads of 50 posts each. That is 2 writes vs 1,000 post reads, about 500:1, so read-heavy, and a precomputed feed plus cache is worth it.

**Q5. A URL shortener does 120,000 redirects/s with a 99% cache hit rate. What load reaches the database?** SCENARIO

1% of 120,000 = 1,200 reads/s, plus about 1,200 inserts/s. A primary with a couple of replicas handles that. Without the cache, 120,000 reads/s would need many replicas.

**Q6. A chat app: which parts are read-heavy and which write-heavy?** SCENARIO

Message sending and storage are write-heavy: a partitioned LSM store keyed by conversation. Profiles, contact lists and settings are read-heavy: cache them. Choose per data type.

**Q7. Why is caching a URL shortener easier than caching a product catalog?** TRICKY

Short links are immutable, so a cached entry never goes stale and invalidation is trivial. Prices and stock change, so the catalog needs TTLs or invalidation on every update.

**Q8. A celebrity with 50 million followers posts. Why does fan-out on write hurt here?** TRICKY

One post becomes 50 million feed writes, a huge burst. Use fan-out on read for celebrities (merge their posts at read time) and fan-out on write for normal users: a hybrid.

Next: SD-D-006 · Storage Estimation backendroadmap.com

# Storage Estimation: Size × Count × Retention × Replicas

*the factor of three most quick answers miss*

Watch the 4-min video

**In one line:** storage = object size × new objects per day × days retained × replication factor, plus metadata, indexes and derived copies, adjusted for growth; the replication factor alone is usually 3×.

## 1 The formula

Users upload **500 million photos a day**, each about **2 MB**. How much storage do you need over five years? The quick answer misses a factor of three or more, and that is the gap between a sensible budget and a very unpleasant surprise.

```
storage = size × objects/day × days × replicas + overhead
```

Write each factor down, round it to an easy number, and multiply step by step. Saying each step out loud lets the interviewer follow and catch a slip early.




**Figure 1.** Four factors multiply into raw storage; overhead comes on top.

## 2 Why multiply by the replication factor

Every byte is stored more than once, so a failed disk or node never loses data.

- **Databases** keep a primary and replicas: one primary plus two replicas is 3 copies.
- **Distributed stores** such as Cassandra or HDFS commonly keep **3 copies**.
- **Object stores** like S3 replicate internally; you don't see the copies, but you pay for that durability in the price per GB.
- **Backups and cross-region copies** come on top, so the real number grows further.

## 3 Don't forget overhead

| Extra | Typical size | Why it exists |
|---|---|---|
| Metadata row | ~500 B to a few KB per object | owner, timestamp, location, tags |
| Indexes on metadata | +50–100% of the table size | fast lookups by owner, time, tag |
| Thumbnails, resized versions | +20–50% per photo | different screens and previews |
| Backups, other regions | +1 copy or more each | disaster recovery |

## 4 Worked example: photos over five years

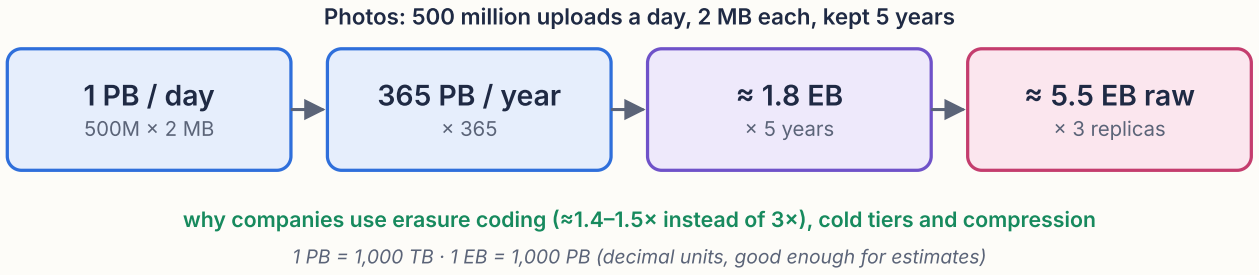



**Figure 2.** Multiply one factor at a time and keep the unit next to every number.

**WORKED EXAMPLE**

500M × 2 MB = 1,000 TB = **1 PB per day**. × 365 ≈ **365 PB per year**. × 5 years ≈ **1.8 EB**. × 3 replicas ≈ **5.5 EB of raw storage**.

Metadata is small next to that: 500M rows × 1 KB = 0.5 TB a day, about 0.9 PB over 5 years before indexes and replicas. But thumbnails at +20–50% add another 1–3 EB of raw storage. Exabytes are huge, which is exactly why companies use **erasure coding** (for example 10 data + 4 parity pieces, 1.4× overhead instead of 3×), **cold storage tiers** and **compression**.

## 5 Interview question: a chat app

**Five-year storage for a chat app?** State the assumptions first: **50M daily users**, each sending **40 messages a day**, about 100 bytes of text plus 100 bytes of metadata per message.

- Messages per day: 50M × 40 = **2 billion**.
- Per day: 2B × 200 B = **400 GB**.
- Five years: 400 GB × 1,825 days ≈ **730 TB**.
- × 3 replicas ≈ 2.2 PB, plus indexes: about **2.5–3 PB**.

Media attachments (photos, voice notes, video) go to object storage and are estimated separately with the same formula. They usually dwarf the text.

## 6 Plan for growth, then look for savings

Data rarely grows linearly. If users double every year, the last year writes as much as all earlier years together, plus a bit more. A flat “year 1 × 5” estimate can be several times too low.

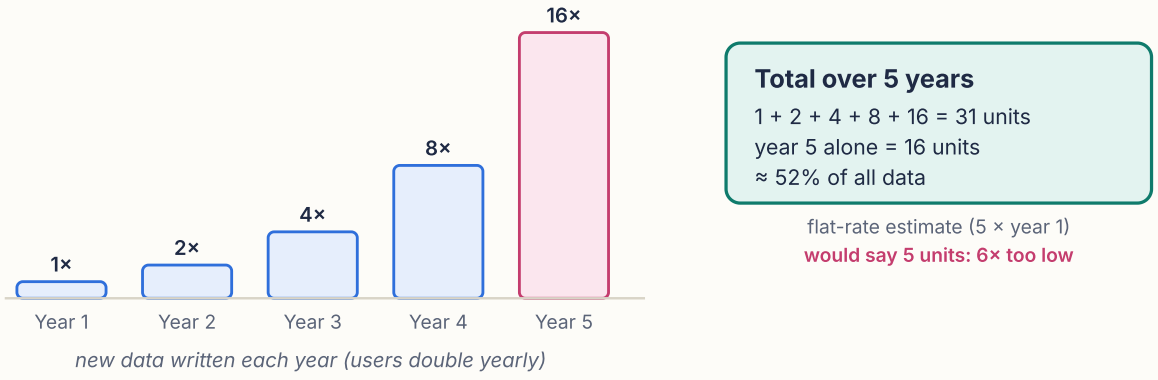



**Figure 3.** With yearly doubling, the fifth year alone is about half of the five-year total.

**Savings:** compress text (chat logs and JSON often shrink several times), move old data to cheaper tiers, expire data you don't need (TTLs, retention policies), and deduplicate identical uploads by content hash.

**COMMON MISTAKES**

- Forgetting the replication factor: a 3× underestimate.
- Ignoring metadata, indexes and thumbnails.
- Mixing bits and bytes, or GB and GiB, mid-calculation; pick decimal units and stay consistent.
- Assuming flat growth for a product that is doubling.
- Storing media in the database instead of object storage.

## 7 Key terms

**Retention**: how long data is kept

**Erasure coding**: data + parity pieces, ~1.4–1.5×

**PB / EB**: $10^{15}$ / $10^{18}$ bytes

**Replication factor**: number of copies, often 3

**Cold tier**: cheap, slow storage for old data

**Derived copy**: thumbnail, resized or transcoded version

**Overhead**: metadata, indexes, derived copies

**Deduplication**: store identical content once

**Object storage**: S3-style store for large blobs

## 8 Interview questions

**Q1. How do you estimate 5-year storage?** BASIC

Object size × new objects per day × days retained × replication factor, then add metadata, indexes and derived copies, and adjust for growth. Round each factor and multiply step by step.

**Q2. Why multiply by the replication factor?** BASIC

Each byte is physically stored several times, usually 3, for durability and availability, and you pay for every copy. Skipping it underestimates by 3×.

**Q3. 500M photos a day at 2 MB, kept 5 years. Raw storage?** SCENARIO

1 PB/day × 365 ≈ 365 PB/year × 5 ≈ 1.8 EB × 3 replicas ≈ 5.5 EB, before thumbnails and metadata.

**Q4. Estimate 5-year storage for a chat app with 50M DAU.** SCENARIO

40 messages each = 2B/day × 200 B = 400 GB/day × 1,825 ≈ 730 TB × 3 ≈ 2.2 PB, plus indexes ≈ 2.5–3 PB. Media goes to object storage, estimated separately.

**Q5. Your estimate is 5 EB. How would you bring the cost down?** SCENARIO

Erasure coding instead of 3 full copies (≈1.4× instead of 3×), cold tiers for old photos, compression, deduplication by content hash, and deleting unused derived sizes.

**Q6. Users double every year. How does that change a 5-year estimate?** TRICKY

Yearly new data goes 1, 2, 4, 8, 16: a total of 31 units, not 5. The final year is about half the total, so size for the last year, not the first.

**Q7. Does S3 mean you can ignore replication?** TRICKY

You don't multiply the bytes yourself, since S3 replicates internally and charges per logical GB. But you still pay for it in the price, and cross-region replication or backups add real copies you do pay for.

Next: SD-D-007 · Bandwidth Estimation backendroadmap.com

# Bandwidth Estimation: Ingress, Egress, Gbps & CDN Offload

*data per second, and why video lives on CDNs*

Watch the 3-min video

**In one line:** bandwidth = requests per second (or concurrent streams) × payload size, converted from bytes to bits, then sized for the peak and reduced by CDN offload; 10,000 streams at 5 Mbps is 50 Gbps of egress.

## 1 Ingress and egress

Your video platform has **10,000 people watching at the same time**, each stream at **5 megabits per second**. Can one data-center link handle that? What will the monthly bill look like? Bandwidth is often the most expensive line in a video platform's budget, so you need to estimate it.

**Bandwidth is data per second. Ingress** is data coming into your system: uploads and API requests. **Egress** is data going out: page loads, downloads and video. For most consumer apps egress is much larger, and cloud providers usually charge for egress, not ingress.

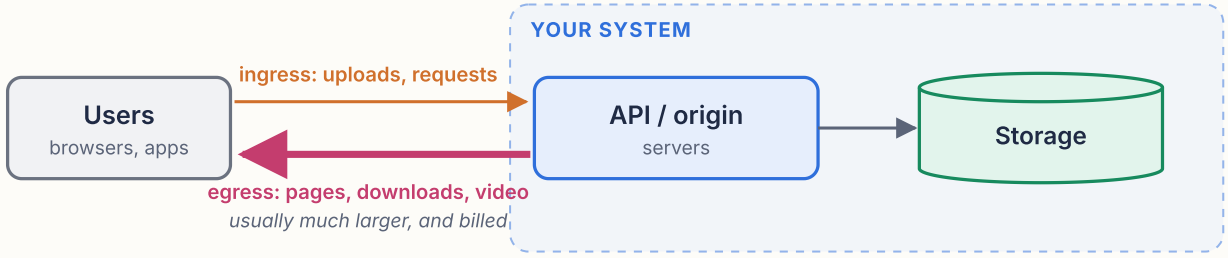
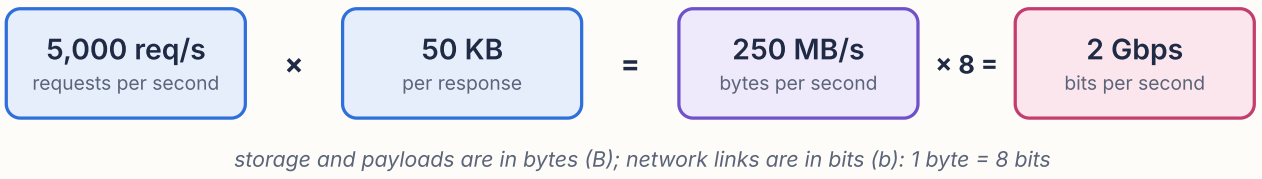



**Figure 1.** Requests are small and come in; responses are large and go out. Egress is the number that drives cost.

## 2 The formula, and the bits-vs-bytes trap

```
bandwidth = requests per second × payload size  ·  Gbps = GB/s × 8
```

For example, 5,000 requests per second × 50 KB per response = **250 MB/s**. Network links are measured in **bits**, so multiply bytes by 8: 250 MB/s × 8 = 2,000 Mbps = **2 Gbps**. For streaming, use concurrent streams × bitrate instead; bitrates are already in bits.




*storage and payloads are in bytes (B); network links are in bits (b): 1 byte = 8 bits*

**Figure 2.** Bytes per second from the app; bits per second for the network. Forgetting the × 8 gives an answer 8 times too small.

## 3 Interview question: 10,000 concurrent streams

**WORKED EXAMPLE**

**Rate:** 10,000 streams × 5 Mbps = 50,000 Mbps = **50 Gbps** of egress.

**Hardware:** a typical server network card is 10 or 25 Gbps. Even at full line rate, 50 Gbps needs 2 × 25 Gbps or 5 × 10 Gbps cards; with headroom, several servers just for the network.

**Monthly volume:** 50 Gbps ÷ 8 = 6.25 GB/s. A 30-day month has about 2.6 million seconds, so 6.25 GB/s × 2.6M s ≈ **16 PB** of traffic a month, "over 15 petabytes". Even at an illustrative 1 cent per GB, that is about $160,000 a month for this one stream of traffic.

## 4 CDN offload

That is why video platforms are dominated by **CDN egress**. The CDN stores popular videos at edge locations near viewers and serves most bytes from there. With a **95% offload**, the origin serves only 5%: **2.5 Gbps instead of 50**. The CDN bill is still large, but much cheaper per GB, and viewers get lower latency because the bytes travel a shorter distance.

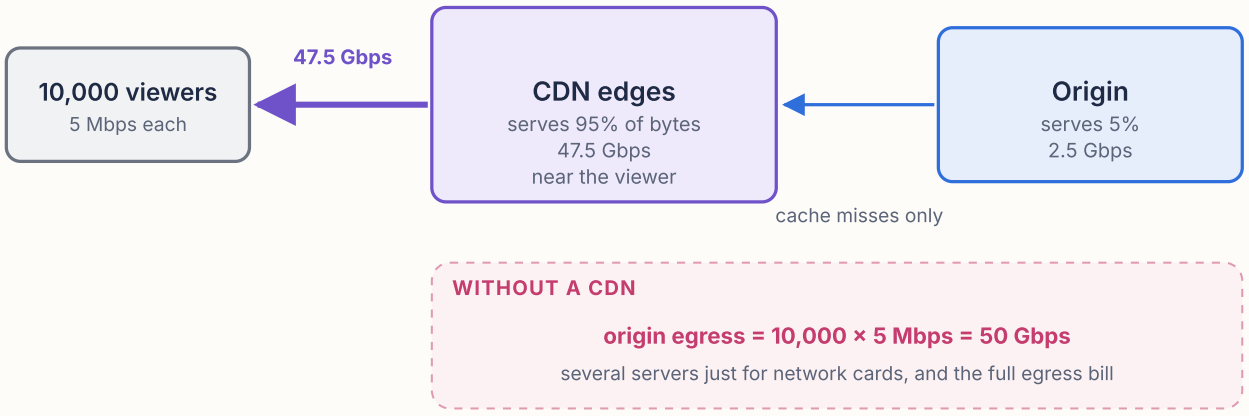



**Figure 3.** With 95% offload, the origin's egress drops from 50 Gbps to 2.5 Gbps; the CDN carries the rest from edges close to viewers.

## 5 Peaks and cross-region transfer

- **Size for the peak, not the average.** Evening prime time can be several times the daily average, and a big live event many times more. If prime time is 3× the average, 50 Gbps average means links and CDN commitments for about 150 Gbps.
- **Cross-region transfer is billed.** Copying data between regions or availability zones costs money. Replicating every upload to three regions multiplies that cost.

## 6 Ways to cut bandwidth

- **Compress responses** with gzip or brotli; JSON and HTML often shrink several times.
- **Modern codecs** for images and video (WebP or AVIF images; H.265, VP9 or AV1 video) send fewer bits for the same quality.
- **Adaptive bitrate** streaming matches the bitrate to the viewer's screen and network: no 4K stream to a phone on 4G.
- **Cache aggressively at the edge.**
- **Paginate** API responses, and **don't send fields** the client never uses.

**COMMON MISTAKES**

- Forgetting to multiply bytes by 8 when comparing with a network link.
- Sizing for the average instead of prime time or a live event.
- Estimating ingress and ignoring egress, which is where the bill is.
- Forgetting cross-region and cross-zone transfer charges.
- Serving video from the origin instead of a CDN.

## 7 Key terms

**Bandwidth**: data per second (Mbps, Gbps)

**Bitrate**: bits per second of a stream

**Peak factor**: peak ÷ average traffic

**Ingress**: data into your system

**CDN offload**: share of bytes served by the CDN

**NIC**: network card, e.g. 10 or 25 Gbps

**Egress**: data out; usually billed

**Origin**: your own servers behind the CDN

**Adaptive bitrate**: stream quality follows the network

## 8 Interview questions

**Q1. How do you estimate bandwidth?** BASIC

Requests per second (or concurrent streams) × payload size, converting bytes to bits, then apply a peak factor and the CDN offload.

**Q2. What is the difference between ingress and egress, and which costs more?** BASIC

Ingress is data coming in (uploads, requests); egress is data going out (pages, video). Egress is usually much larger, and clouds usually bill egress, not ingress.

**Q3. 5,000 RPS with 50 KB responses. What bandwidth?** SCENARIO

5,000 × 50 KB = 250 MB/s × 8 = 2 Gbps. That fits one 10 Gbps card, but plan for peak and use several servers for redundancy.

**Q4. 10,000 concurrent streams at 5 Mbps. What egress, and what do you do about it?** SCENARIO

50 Gbps, about 16 PB a month. Push it through a CDN: at 95% offload the origin serves only 2.5 Gbps.

**Q5. Average egress is 20 Gbps and prime time is 4× average. What do you provision?** SCENARIO

Size links and CDN commitments for about 80 Gbps, plus headroom for a live event, not 20 Gbps.

**Q6. Your API responses are 2 MB each and bandwidth is high. What can you change besides adding links?** TRICKY

Compress with gzip or brotli, paginate, remove unused fields, cache at the edge. A 2 MB JSON response often shrinks to a few hundred KB compressed.

**Q7. You replicate every upload to three regions. What happens to cost?** TRICKY

Each upload is transferred to two other regions, and cross-region transfer is billed, so that transfer cost is paid twice per upload, on top of 3× the storage. Replicate only what needs it.

Next: SD-D-008 · Cache Sizing backendroadmap.com

# Cache Sizing: Working Set, the 80/20 Rule & Redis Nodes

*how much memory, and how many nodes*

Watch the 3-min video

**In one line:** estimate the working set (data actually read in a period), cache its hot fraction (80/20), multiply by entry size plus overhead, then add headroom and replicas to get a node count, and check throughput per node.

## 1 Too small or too large

Your URL shortener serves **1 billion redirects a day**, and you want Redis in front of the database. How much memory should the cache have? 10 GB? A terabyte? Guess too small and the **hit rate collapses**, sending traffic back to the database. Guess too large and you **pay for memory nobody uses**.

## 2 Start with the working set

The **working set** is the data actually accessed during a period, such as a day. It is usually far smaller than the total data. Years of links exist, but most redirects hit links created recently, or a few viral ones. You only need to cache the working set, not everything.

Then apply the **80/20 rule**: about 20% of items get 80% of the traffic. Real traffic is often even more skewed. So caching the hottest 20% of what is read each day usually gives a high hit rate. Measure the real distribution when you can.

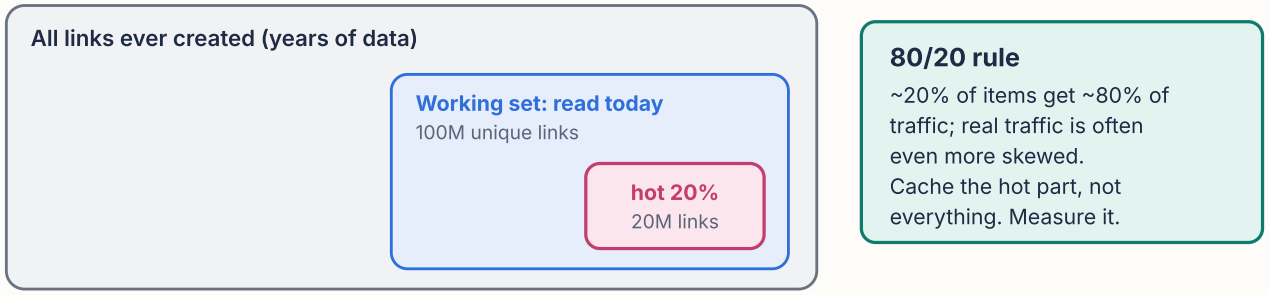



**Figure 1.** Total data ⊃ working set (read today) ⊃ hot 20%. The cache only needs the inner boxes.

## 3 Size per entry: value plus overhead

The value is only part of the cost. A short code plus a long URL might be about **500 bytes**. Redis adds per-key overhead: pointers, expiry data and allocator fragmentation, often **50–100 bytes per key**, sometimes more. For small entries, a safe estimate is to **add about 50%**.

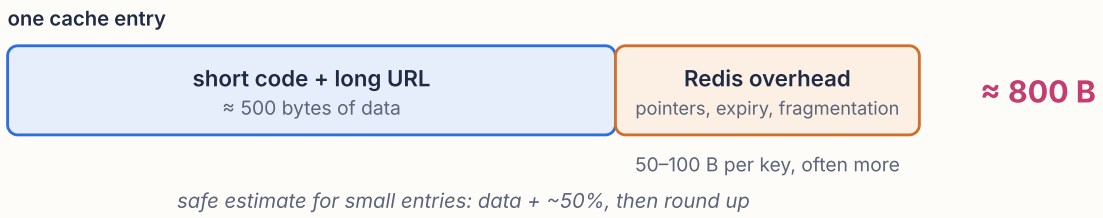



**Figure 2.** 500 bytes of data becomes about 750 B with overhead; round up to 800 B for easy maths.

## 4 Worked example

**WORKED EXAMPLE: URL SHORTENER CACHE**

**Working set:** 1 billion redirects a day, but say only **100 million unique links** are read.

**Hot fraction:** cache 20% = **20 million entries**.

**Size:** 500 B + overhead, round to 800 B. 20M × 800 B = **16 GB**.

**Upper bound:** if we had sized for 20% of *all* 1 billion reads (200M entries), that is 200M × 800 B = **160 GB**. Still very manageable, so size for 160 GB to be safe.

## 5 From memory to nodes

**Don't fill memory to the top.** Leave about **25% headroom** for spikes, fragmentation and snapshots (Redis forks to write RDB snapshots, and pages changed during the fork are copied). A 64 GB node gives about 48 GB usable. 160 GB ÷ 48 GB = 3.3, so **4 primaries**. Add one replica each for availability: **8 nodes in total**.

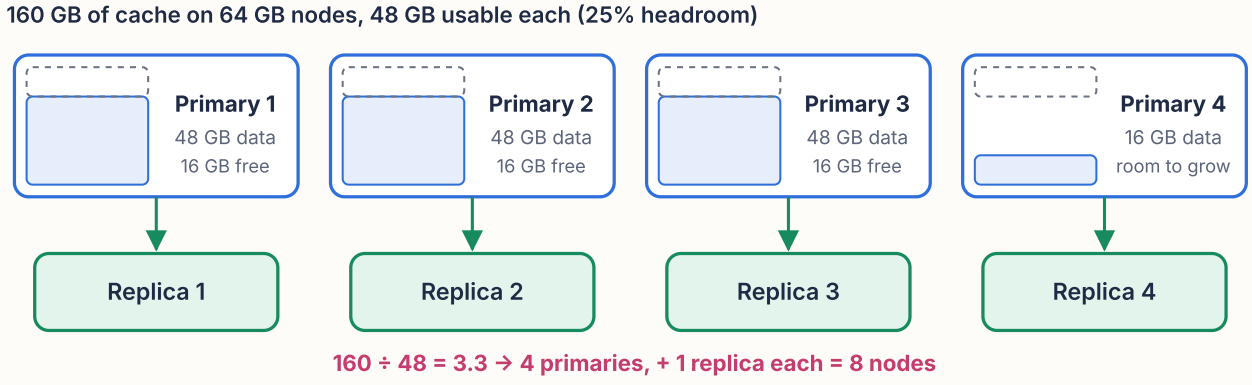



**Figure 3.** Four primaries hold the 160 GB with headroom; each has a replica that takes over if it fails.

**Throughput matters too.** One Redis node handles around **100,000 operations per second**. 1 billion redirects a day is about 11,600 per second on average; even at a 3× peak (≈35,000/s) four primaries are far from their limit. If peak ops were 1 million per second, you would need at least 10 primaries for throughput, whatever the memory says.

## 6 Validate with the hit rate

Track the **hit rate** in production. If it drops, either the working set grew or your eviction policy is wrong. Use an eviction policy such as **least recently used** (Redis `maxmemory-policy allkeys-lru`) and **time-to-live** values, so cold data leaves the cache automatically.

**COMMON MISTAKES**

- Sizing the cache for the whole dataset instead of the working set.
- Counting only the value bytes and ignoring per-key overhead.
- Filling nodes to 100%: no room for spikes, fragmentation or snapshots.
- Checking memory but not operations per second per node.
- No eviction policy or TTLs, so the cache fills with cold data.

## 7 Key terms

**Working set**: data actively accessed in a time window

**Per-key overhead**: metadata Redis keeps per key

**Eviction policy**: what to drop when full, e.g. LRU

**80/20 rule**: ~20% of items get ~80% of traffic

**Fragmentation**: memory lost between allocations

**TTL**: time-to-live; key expires

**Hit rate**: share of reads served from cache

**Headroom**: spare memory, ~25%

**Replica**: copy of a primary for failover

## 8 Interview questions

**Q1. What is a working set?** BASIC

The subset of data actively accessed over a time window, such as a day. It is usually far smaller than the total dataset, and it is what you size a cache for.

**Q2. How do you size a cache?** BASIC

Estimate the working set, apply 80/20 to cache the hot fraction, multiply by entry size plus overhead, add ~25% headroom and replicas to get the node count, and check ops/s per node.

**Q3. 100M unique links read per day, 800 B per entry. How big a cache?** SCENARIO

20% of 100M = 20M entries × 800 B = 16 GB. A single 64 GB node holds it, plus a replica for availability.

**Q4. You need 160 GB of cache on 64 GB nodes. How many nodes?** SCENARIO

Use about 48 GB per node (25% headroom): 160 ÷ 48 = 3.3, round up to 4 primaries. One replica each: 8 nodes.

**Q5. The hit rate drops from 95% to 70% over a month. Why, and what do you do?** SCENARIO

The working set grew (more traffic, more unique keys) or eviction is wrong (no LRU, TTLs too long or too short). Check evictions and memory, add nodes or fix the policy.

**Q6. Memory says 2 nodes are enough, but peak is 500,000 ops/s. How many?** TRICKY

At ~100,000 ops/s per node, you need at least 5 primaries for throughput, more with headroom. Take the larger of the memory count and the throughput count.

**Q7. Why not fill each Redis node to 100% of RAM?** TRICKY

Spikes, fragmentation and snapshot forks need extra memory. At 100% the node evicts aggressively or runs out of memory. Keep about 25% free.

Next: SD-D-009 · How Many Servers Do You Need? Capacity, 60% Target & N+2 backendroadmap.com

# How Many Servers Do You Need? Capacity, 60% Target & N+2

*measured capacity, a safe target, and room for failures*

Watch the 4-min video

**In one line:** servers = peak RPS ÷ (measured per-instance capacity × a 60–70% utilization target), then add redundancy (N+1 or N+2) and spread across availability zones; and check that the databases and caches behind them can take the same load.

## 1 Three numbers, not one

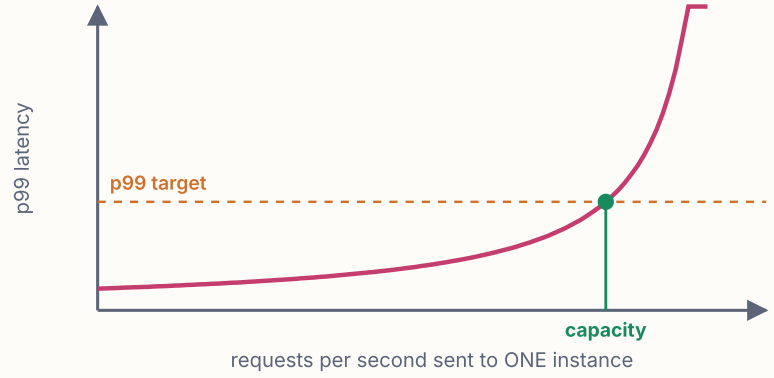
Your API must handle **12,000 requests per second at peak**. How many servers, or Kubernetes pods, do you need? Dividing by a number you heard somewhere is not enough. You need three things:

1. the **capacity of one instance**, measured;
2. a **safe utilization target**;
3. **room for failures**.

```
instances = peak RPS ÷ (capacity per instance × target) + redundancy
```

## 2 Measure per-instance capacity

Run a **load test** against one instance and increase traffic until latency, such as the **99th percentile (p99)**, crosses your target. That throughput is the instance's capacity. Rules of thumb vary hugely: a simple cached endpoint might handle thousands of requests per second per core, a heavy endpoint only tens.




**Measure, don't guess**

Ramp traffic on one instance until p99 crosses the target. That RPS is its capacity. Cached endpoint: thousands of RPS per core. Heavy one: tens.

**Figure 1.** Capacity is the highest RPS at which p99 still meets the target, not the point where the server falls over.

## 3 CPU-bound or I/O-bound?

| | CPU-bound | I/O-bound |
|---|---|---|
| Examples | image resizing, encryption, model inference | calling databases and other services |
| Where time goes | computing | waiting on the network or disk |
| Scales with | cores: more requests need more CPUs | concurrency: async I/O lets one instance hold many requests |
| Real limit | CPU | often the database or the connection pool |

For I/O-bound services, Little's Law helps: an instance doing 1,000 RPS at 50 ms per request holds about 1,000 × 0.05 = 50 requests in flight, so it needs a pool and connections for at least 50.

## 4 Never plan to run at 100%

Latency rises sharply as utilization nears the limit, because requests queue up. A common target is **60–70% at peak**. That leaves headroom for spikes, garbage-collection pauses, noisy neighbours and uneven load balancing.

**WORKED EXAMPLE: 12,000 RPS**

Each pod handles **1,000 RPS** at its limit. At a **60% target**, each pod should take 600. 12,000 ÷ 600 = **20 pods**.

## 5 Add redundancy and zones

- **N+1**: you can lose any one instance and still handle peak load.
- **N+2**: you survive a failure *during* a deployment or maintenance, when one instance is already out.
- **Availability zones**: spread instances across at least **3 zones**. If one zone fails, the other two must carry the whole load, which can mean **50% more instances**.

So 20 pods becomes about **22 with N+2**, or **30 for full zone tolerance** (10 per zone, so any two zones still have the 20 you need).

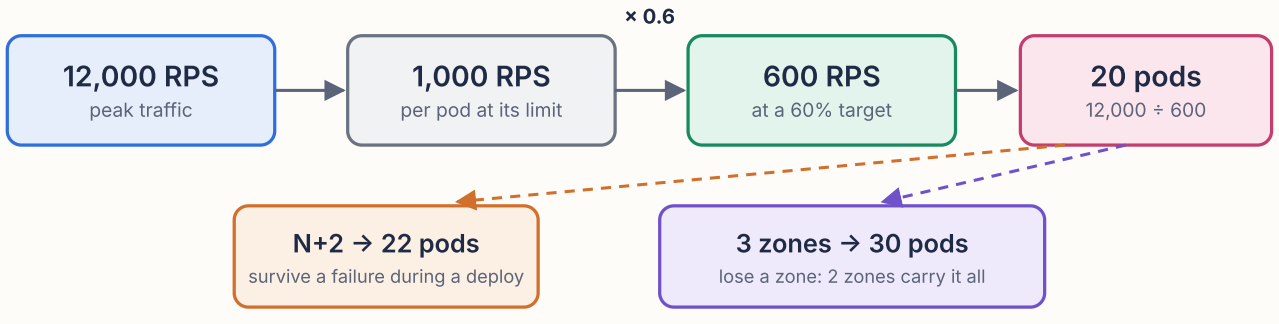



**Figure 2.** From peak traffic to a pod count, then two ways to add safety: N+2 for instance failures, and 50% extra for losing a whole zone.

## 6 Interview question: 20,000 RPS

**How many servers for 20,000 requests per second?** Say one server handles **2,000 RPS** at its limit, and we target 60%, so **1,200** each. 20,000 ÷ 1,200 ≈ 16.7, round up to **17**. Add N+2: **19**. Spread across three

zones, rounding up to 7 per zone: **21 servers**.

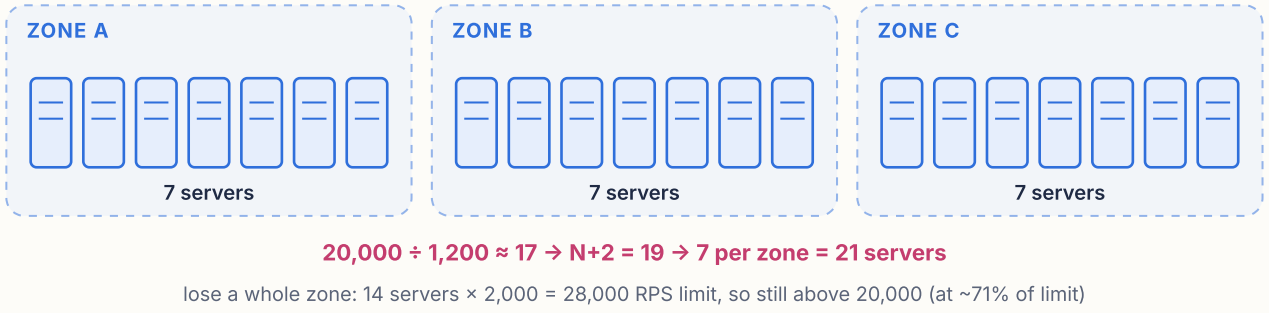



**Figure 3.** 21 servers over three zones. Losing a zone leaves 14, which still stay under their limit, though above the 60% target until autoscaling adds more.

**Remember the dependencies.** The database and caches behind the servers must handle the same load. 20,000 RPS with two queries per request is 40,000 queries per second on the data tier, unless the cache absorbs most of them.

COMMON MISTAKES

- Using a capacity number from a blog instead of a load test.
- Planning to run at 90–100% utilization.
- Forgetting redundancy, so one failure or one deploy means overload.
- Putting all instances in one zone.
- Scaling the app tier and forgetting the database and connection pool behind it.

## 7 Key terms

**Per-instance capacity**: max RPS while p99 meets target

**Load test**: ramp traffic to find capacity

**Utilization target**: planned busy share, 60–70%

**CPU-bound**: limited by computation

**I/O-bound**: limited by waiting on I/O

**N+1 / N+2**: spare instances for 1 / 2 failures

**Availability zone**: isolated data center in a region

**Headroom**: spare capacity for spikes

**p99**: 99% of requests are faster

## 8 Interview questions

**Q1. How many servers do you need for 20,000 RPS?** BASIC

Measure capacity with a load test (say 2,000 RPS), apply a 60% target (1,200), divide: about 17. Add N+2 (19) and spread across three zones: about 21.

**Q2. Why target 60–70% utilization instead of 100%?** BASIC

Latency rises sharply near the limit because requests queue. Headroom covers spikes, GC pauses, noisy neighbours and uneven load balancing.

**Q3. How do you find one instance's capacity?** BASIC

Load-test a single instance, ramping traffic until p99 latency crosses the target. The RPS at that point is its capacity.

**Q4. 12,000 RPS peak, 1,000 RPS per pod at its limit. How many pods?** SCENARIO

At 60%, 600 per pod: 12,000 ÷ 600 = 20. N+2 gives 22; full zone tolerance across 3 zones gives 30.

**Q5. Your service mostly calls a database. What limits it, and how do you scale it?** SCENARIO

It is I/O-bound: the limit is often the database or connection pool, not CPU. Use async I/O, size pools with Little's Law, and make sure the database can take the total load before adding app servers.

**Q6. Why can surviving a zone failure need 50% more instances?** TRICKY

With 3 zones, losing one leaves two-thirds of the fleet. To still serve full load, each zone must hold half of what you need: 20 needed → 10 per zone → 30 total.

**Q7. You added servers but latency did not improve. Why?** TRICKY

The bottleneck is elsewhere: the database, a cache, a connection pool or a downstream service. Find the saturated resource before scaling the app tier.

Next: SD-D-010 · Growth Projections

# Growth Projections

*compound growth, runway and when to shard*

Watch the 3-min video

**In one line:** project each resource's peak with compound growth, divide into its safe limit to get the runway, and start each big change (replicas, caching, sharding) one lead time before that runway runs out.

## 1 Why you project growth

Your database runs at 40% CPU today and traffic grows fast. Will it survive next year? If you only notice at 100%, you are months too late: **sharding a database takes a team a quarter or more**. A growth projection is how you know when to *start*, not just when you will break.

## 2 Growth compounds

Growth multiplies; it does not add. Ten percent a month is not 120% a year. It is **$1.1^{12} \approx 3.1\times$** in one year. The yearly figure is the **compound annual growth rate (CAGR)**. Three times a year, kept up for three years, is 3 × 3 × 3 = **27×**.

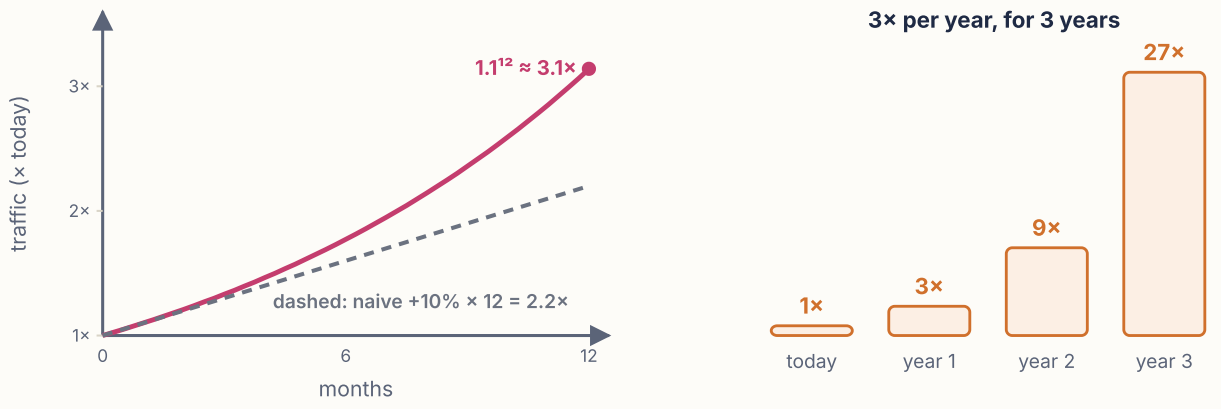



**Figure 1.** Left: 10% a month compounds to about 3.1× in a year, well above the naive 2.2×. Right: 3× a year for three years is 27×.

$$load(t) = load\ today \times (1 + g)^t \quad \cdot \quad monthly\ g = CAGR^{1/12} - 1$$

Going the other way is handy too: "doubles every year" means about **5.9% a month** ($2^{1/12} \approx 1.059$).

## 3 Project per resource, then compute the runway

Start from today's **measured peak**, not the average. Apply the growth rate and calculate the load at future dates: 6 months, 1 year, 2 years, 3 years. Do it **per resource**, because requests per second, storage, database writes and bandwidth often grow at different rates. Storage, for example, only ever accumulates.

Then compare each projection with that resource's capacity and compute the **runway**: how long until it reaches its *safe* limit.

```
runway = log(safe limit ÷ current usage) ÷ log(growth factor per period)
```

Or simply step forward month by month in a spreadsheet. **The resource with the shortest runway is your next bottleneck.**

**WORKED EXAMPLE: THE RUNWAY FORMULA**

A database is at **40% CPU**, traffic grows **10% a month**, and the safe limit is **70%**. Runway = log(70 ÷ 40) ÷ log(1.1) = log(1.75) ÷ log(1.1) ≈ 0.56 ÷ 0.095 ≈ **5.9 months**. Less than half a year, while the fix may take a quarter.

## 4 The workbook case: when to start sharding

The primary database is at **30%** of its write capacity, and writes **double every year**. The trigger for action is **60%**, not 100%, because you need time and headroom for spikes and failover.

- 30% doubles to 60% in exactly **12 months**.
- The sharding project needs **6 months**, so it must start within **6 months**.
- The plan: start the sharding work now, aim to finish before month 12, and treat about **month 18** (≈ 85%, 60% plus a safety buffer) as the absolute latest.

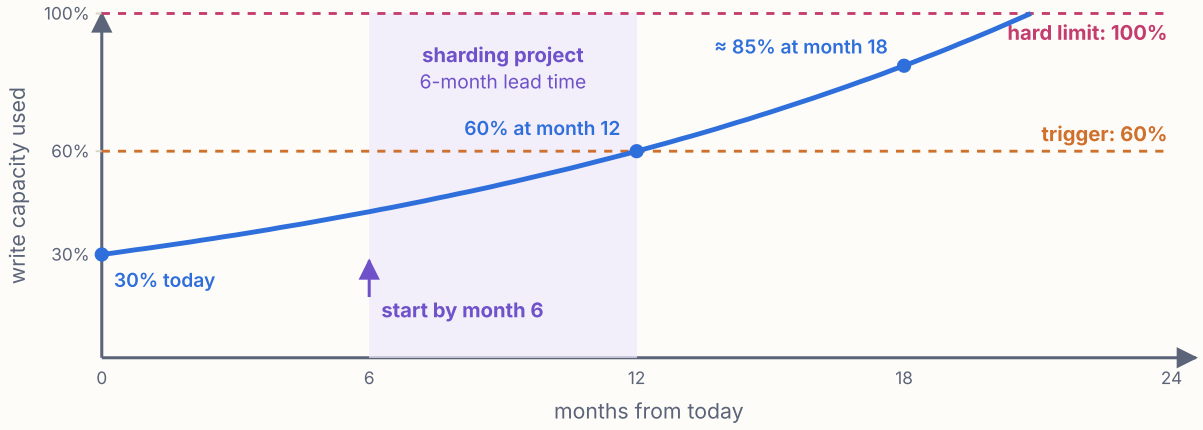



**Figure 2.** Write load doubling yearly from 30%. It crosses the 60% trigger at month 12, so a 6-month project must start by month 6. Waiting for 100% (about month 21) leaves no time.

## 5 Plan re-architecture thresholds in advance

For each limit, write down two things before you need them: a **trigger** (like 60% of capacity) and the **action** it starts. A typical database ladder: add read replicas, scale up, add caching, partition tables, then shard. Each step has a lead time, and the bigger the change, the earlier you must begin.




**Figure 3.** A re-architecture ladder. Lead times are typical ranges, not rules: measure your own team's.

**INTERVIEW ANSWER: PLAN FOR 3× GROWTH NEXT YEAR**

- **Multiply** today's peak for every resource by 3, then add headroom.
- **Check what breaks first.** App servers may just need autoscaling; the database may need replicas or sharding; the cache may need more nodes.
- **Order the work** by runway and lead time: the database usually goes first, because it has the longest lead time.
- **Validate** with load tests at 3× today's traffic.

**COMMON MISTAKES**

- Adding growth linearly: 10% a month is 3.1× a year, not 2.2×.
- Projecting from the average instead of the measured peak.
- Using one growth rate for everything; storage, writes and bandwidth diverge.
- Setting the trigger at 100%. The trigger must leave room for the lead time of the fix.
- Planning only the cheap stateless tier and leaving the database, the slowest change, for last.

## 6 Key terms

**Compound growth**: growth that multiplies each period: $(1 + g)^t$

**CAGR**: compound annual growth rate

**Peak load**: highest measured load, the base for projections

**Capacity**: the most a resource can sustain

**Runway**: time until a resource hits its safe limit

**Trigger threshold**: the level (e.g. 60%) that starts the next step

**Lead time**: how long a change takes from start to ready

**Headroom**: spare capacity kept for spikes and failures

## 7 Interview questions

**Q1. Traffic grows 10% a month. How much bigger is it in a year?** BASIC

$1.1^{12} \approx 3.1\times$, not 2.2×. Growth compounds. In two years it is about 9.8×.

**Q2. What is runway, and how do you compute it?** BASIC

Time until a resource reaches its safe limit. Runway = log(limit ÷ current) ÷ log(growth factor). At 40% CPU, 10% monthly growth and a 70% limit: log 1.75 ÷ log 1.1 ≈ 5.9 months.

**Q3. Why set the trigger at 60% instead of 100%?** BASIC

Big fixes have lead times. Writes doubling yearly go from 60% to 100% in about 9 months; a sharding project can take 6 or more. The gap also absorbs spikes and failover load.

**Q4. The primary is at 30% of write capacity and writes double yearly. Sharding takes 6 months. When do you start?** SCENARIO

30% reaches the 60% trigger at month 12. Start within 6 months, ideally now. By month 18 it is near 85%: the absolute latest.

**Q5. How do you plan for 3× growth next year?** SCENARIO

Multiply each resource's peak by 3 and add headroom. Find what breaks first: app servers autoscale, the database needs replicas or sharding, the cache needs nodes. Order the work by runway and lead time, and load-test at 3×.

**Q6. Requests grow 2× a year but storage grows 4× a year. Which do you watch?** SCENARIO

Both, separately. Storage compounds on top of what you already store, so it can have the shorter runway even though request rates look fine.

**Q7. A doubling-yearly service is at 50% today. Is that safe for 2 years?** TRICKY

No. It reaches 100% at month 12 and 200% at month 24. With a 60% trigger, the runway is only log(1.2) ÷ log(2) ≈ 0.26 years, about 3 months.

**Q8. Why does the database usually go first in a growth plan, even if app servers break earlier?** TRICKY

App servers are stateless: autoscaling them takes minutes. Database changes like sharding take months and move data, so their lead time dominates the schedule.

Next: SD-D-011 · Back-of-the-Envelope Estimation backendroadmap.com

# Back-of-the-Envelope Estimation

*the full URL shortener drill, in five minutes*

Watch the 4-min video

**In one line:** state assumptions, then derive traffic, storage, bandwidth, cache and servers in order, rounding hard and tying each number to a design decision: for a URL shortener that is ~1k writes/s, ~100k reads/s, ~90 TB, ~20 GB of cache and ~100 servers.

## 1 The drill: five steps, out loud

In a real interview you get about **five minutes** for estimation, on a whiteboard, with someone watching. You need a fixed order so you never stall. This lesson runs the whole drill for a URL shortener, using every tool from this section: users, traffic, peak, storage, bandwidth, cache and servers.

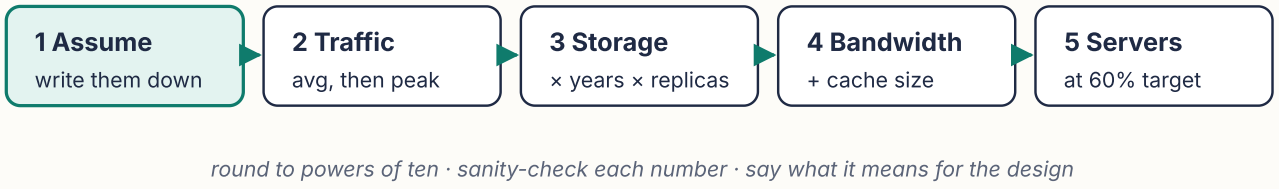



**Figure 1.** The order to work in. Each step feeds the next.

## 2 Step 1: state the assumptions, and write them down

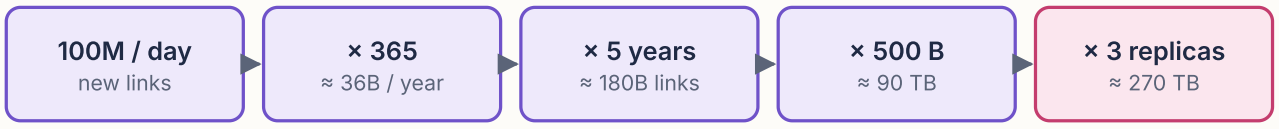
| Assumption | Value |
|---|---|
| New short links per day | 100 million |
| Read : write ratio | 100 : 1 |
| Retention | 5 years |
| Size of each record | ~500 bytes |
| Size of each redirect response | ~500 bytes |
| Peak vs average | 3× |

Written assumptions let the interviewer correct you early, and let you redo the math fast if they change one.

## 3 Steps 2 and 3: traffic and storage

**Traffic.** A day has 86,400 seconds; round it to $10^5$. Writes: 100M ÷ $10^5$ = **~1,000 writes/s**. Reads are 100× that: **~100,000 redirects/s** on average. At 3× peak: **3,000 writes/s** and **300,000 reads/s**.

**Storage.** 100M links/day × 365 ≈ 36 billion a year, so about 180 billion over 5 years. Times 500 bytes is **~90 TB**. With 3 replicas, **~270 TB**.




270 TB is far beyond one server: a partitioned key-value store, keyed by short code

*a 7-character base62 code gives $62^7 \approx 3.5$ trillion codes, plenty for 180 billion links*

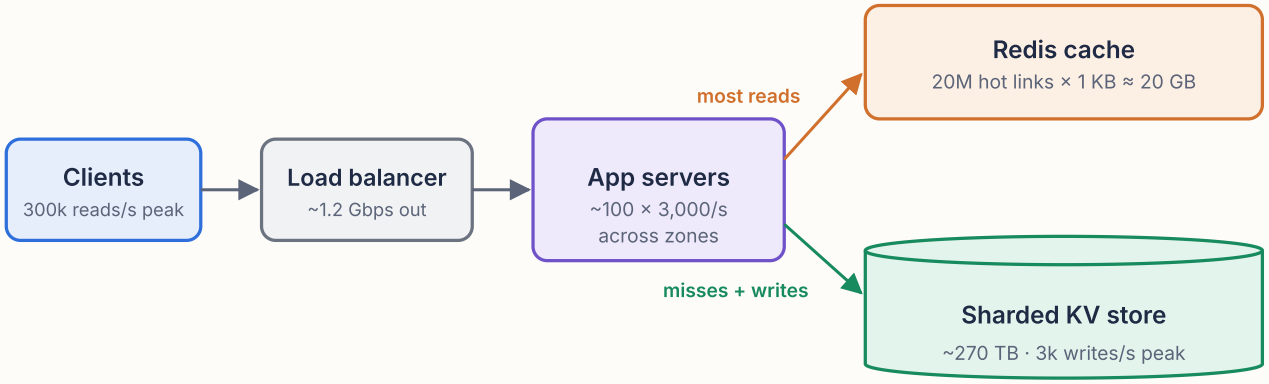
**Figure 2.** The storage chain. Keep one significant figure at each step; the answer only needs the right order of magnitude.

## 4 Steps 4 and 5: bandwidth, cache, servers

**Bandwidth.** Peak reads 300,000/s × 500 bytes = **~150 MB/s**, about **1.2 Gbps** of egress (× 8 bits per byte). Fine for a cluster.

**Cache.** Assume the daily set of distinct links read is about 100 million and 20% of them are hot: **20 million entries**. At about 1 KB each with key and overhead, that is **~20 GB of Redis**. Small enough for a few nodes.

**Servers.** If one app server handles 5,000 redirects/s and you target 60% utilization, plan for 3,000 each. 300,000 ÷ 3,000 = **~100 servers**, plus redundancy across zones. Most redirects hit the cache, so the database sees far fewer reads.




*every box carries the number that justified it*

**Figure 3.** The design the numbers point to: cache in front of a sharded key-value store, about 100 stateless app servers.

**WORKED EXAMPLE: THE SUMMARY TABLE TO LEAVE ON THE BOARD**

| Quantity | Estimate | So the design needs... |
|---|---|---|
| Writes | ~1k/s avg · 3k/s peak | modest; any KV store copes |
| Reads | ~100k/s avg · 300k/s peak | a cache in front of storage |
| Storage (5 years) | ~90 TB · 270 TB replicated | sharding / partitioned KV |
| Egress at peak | ~150 MB/s ≈ 1.2 Gbps | nothing special |
| Cache | ~20 GB | a few Redis nodes |
| App servers | ~100 at 60% | stateless, spread across zones |

## 5 How to do it well

- **Round aggressively.** Use powers of ten and keep one significant figure. 86,400 becomes $10^5$; 36.5 billion becomes 36 billion.
- **Sanity-check every result.** Does 90 TB for five years of short links feel right? 180 billion rows of 500 bytes: yes.
- **Say what each number means.** "This needs sharding." "This fits in memory." A number with no consequence is wasted time.
- **Finish with a summary table** so the interviewer sees everything at a glance.

COMMON MISTAKES

- Starting to compute before stating assumptions.
- Sizing servers from the average instead of the peak.
- Forgetting replication (×3) in storage.
- Mixing bytes and bits: 150 MB/s is 1.2 Gbps, not 150 Mbps.
- Spending ten minutes on precise arithmetic the design does not need.

## 6 Key terms

**Back-of-the-envelope**: quick, rounded estimate to size a design

**Peak factor**: peak ÷ average traffic; here 3×

**Hot set**: the share of data most requests touch

**Read : write ratio**: reads per write; here 100 : 1

**Egress**: data sent out to clients

**Target utilization**: planned load per server, e.g. 60%

## 7 Interview questions

**Q1. 100 million new links a day. Writes per second?** BASIC

100M ÷ $10^5$ seconds ≈ 1,000 writes/s on average, about 3,000 at a 3× peak. (Exact: 100M ÷ 86,400 ≈ 1,160.)

**Q2. How much storage for 5 years at 500 bytes per link?** BASIC

100M × 365 × 5 ≈ 180 billion links × 500 B ≈ 90 TB, or about 270 TB with 3 replicas.

**Q3. Peak reads are 300k/s at 500 bytes. What bandwidth?** BASIC

300,000 × 500 B = 150 MB/s, × 8 = 1.2 Gbps of egress.

**Q4. How big is the cache?** SCENARIO

If 20% of a 100-million daily link set is hot, that's 20 million entries at about 1 KB each: ~20 GB, a few Redis nodes.

**Q5. How many app servers?** SCENARIO

One server does 5,000 redirects/s; at a 60% target, 3,000. 300,000 ÷ 3,000 = 100 servers, spread across zones for redundancy.

**Q6. The interviewer says reads are 1,000 : 1, not 100 : 1. What changes?** SCENARIO

Reads become ~1M/s average and 3M/s peak; egress ~12 Gbps; about 1,000 app servers. Storage and writes do not change. The cache hit rate and a CDN now matter much more.

**Q7. Why is the database not the bottleneck at 300k reads/s?** TRICKY

The cache absorbs most redirects. At a 90% hit rate only ~30k reads/s reach storage, spread over many shards. The database's real problem is its size, 270 TB.

**Q8. Why round 86,400 to 100,000?** TRICKY

It makes mental math instant and errs about 15% low, which is well inside the error of the assumptions. Say that you rounded, and keep going.

Next: SD-D-012 · Architecture Diagrams backendroadmap.com

# Architecture Diagrams

*sync vs async, read and write paths, C4*

Watch the 3-min video

**In one line:** lay boxes out left to right in layers, label every box and arrow, draw sync calls solid and async messages dashed through a visible queue, split read and write paths, and pick the right zoom level (C4).

## 1 A layout anyone can follow

You finish your estimates and pick up the marker. Five minutes later the board is a tangle of boxes and crossing arrows, and even you can't say which request goes where. A clear diagram is half of a system design interview.

Lay it out **from left to right, in layers**:

- **Clients** on the left: web, mobile, other services.
- The **edge**: DNS, CDN, and the load balancer or API gateway.
- The **services** in the middle.
- The **data stores** on the right: databases, caches, object storage, search.

Label every box with what it *is* ("Orders DB, Postgres", not "DB") and every arrow with what *flows* along it.

## 2 Sync and async look different

Use **solid arrows** for synchronous calls, where the caller waits for a response, like HTTP or gRPC. Use **dashed arrows** for asynchronous flows, like messages through a queue or a Kafka topic, where the sender doesn't wait. Draw the queue as **its own box**, so everyone sees where work can pile up. Number the steps of important flows so you can walk through them in order.

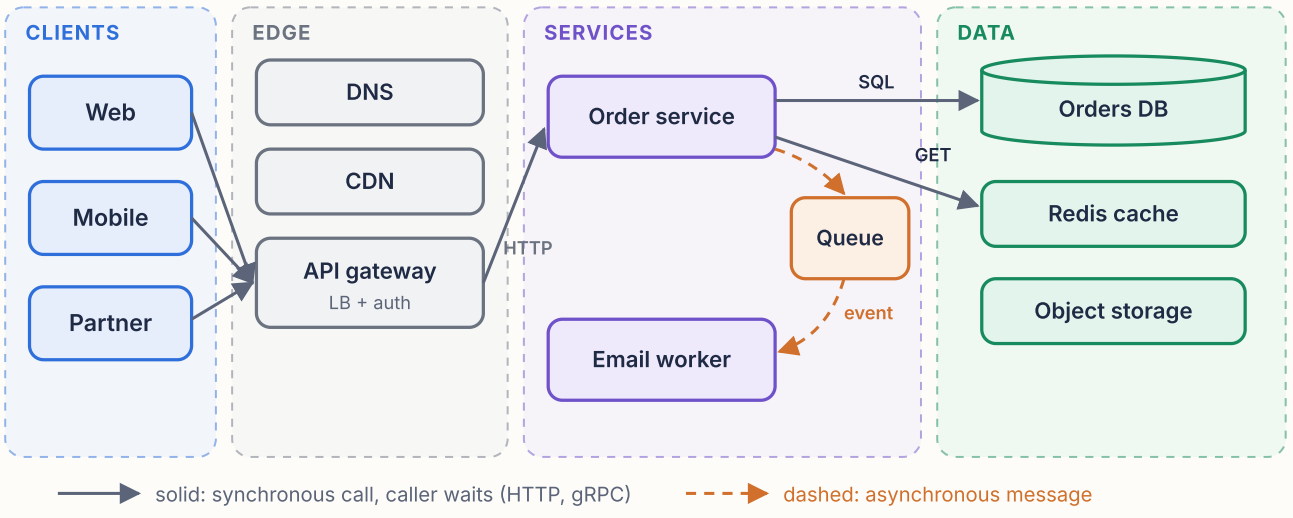



**Figure 1.** Four layers, left to right. The order service answers the client synchronously, and hands the email work to a queue (dashed) that a worker drains later.

## 3 Separate the read path from the write path

Reads and writes often use different components and scale differently. Drawing them separately, or in different colors, makes the design much easier to follow and discuss.

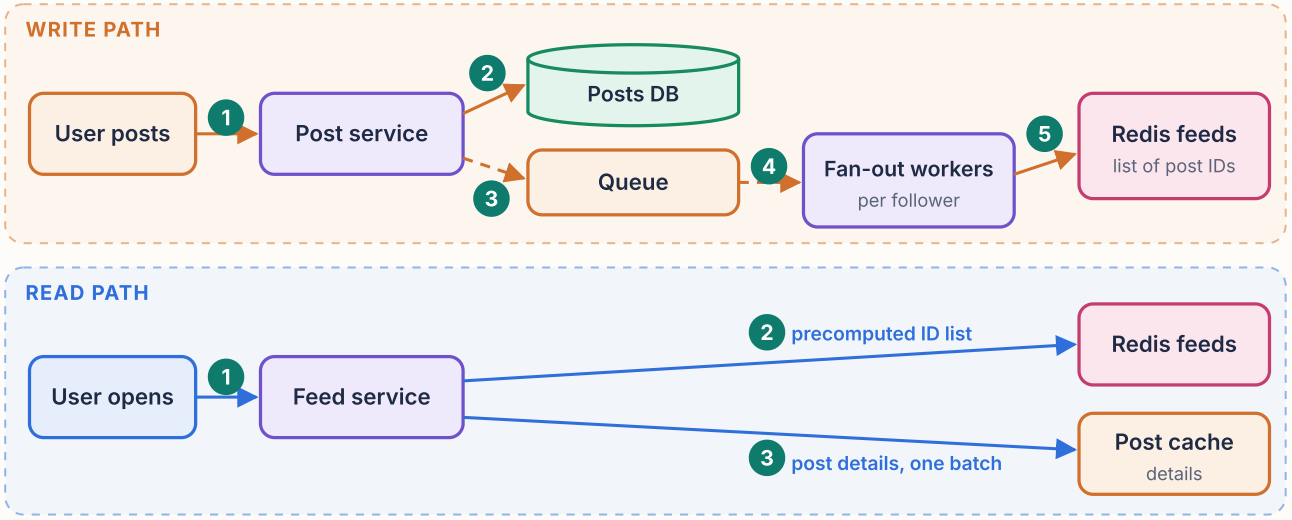



**Figure 2.** A news feed. Writes do the heavy work up front (fan-out on write); reads only touch Redis and a cache.

**WORKED EXAMPLE: THE NEWS FEED**

**Write path.** (1) A user posts. (2) The post service stores it in the posts database and (3) publishes an event to a queue. (4) Fan-out workers consume it and (5) push the post ID into each follower's feed list in Redis.

**Read path.** (1) The user opens the app. (2) The feed service reads their precomputed list from Redis, then (3) fetches post details from a cache. No heavy database queries on the read path.

## 4 Sequence diagrams for one important request

An architecture diagram shows *what* talks to what. A **sequence diagram** shows *when*. Participants are columns: client, gateway, service, cache, database. Time flows downward, and each call and response is a horizontal arrow. It is ideal for a checkout or a login, because you can show retries and timeouts explicitly.

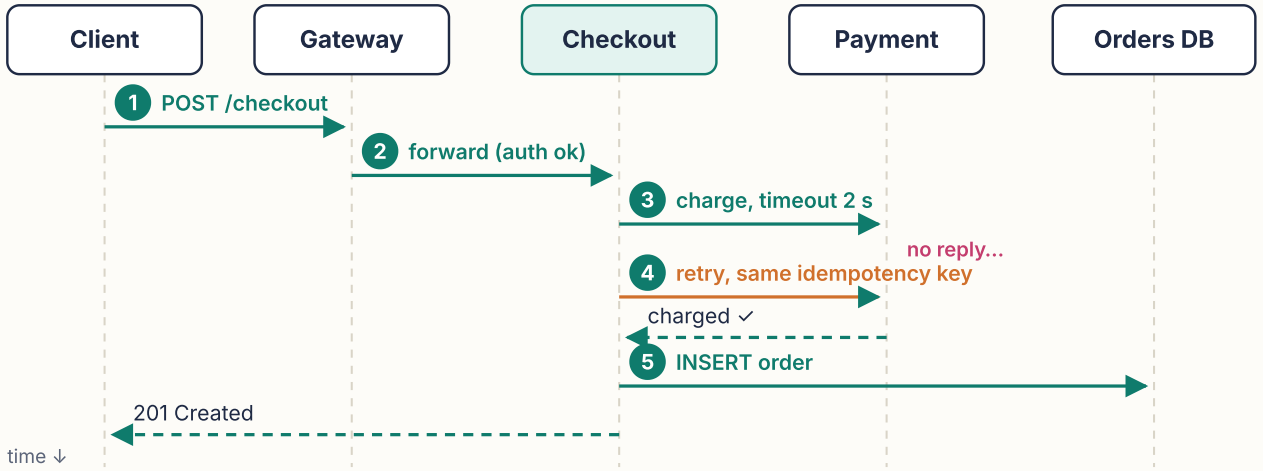



**Figure 3.** Checkout as a sequence. The payment call times out after 2 s and is retried with the same idempotency key, so the customer is charged once.

## 5 C4: four levels of zoom

The **C4 model**, from Simon Brown, gives you levels of zoom so each picture answers one question:

- **Level 1, Context:** your system as one box, with its users and the external systems it talks to.
- **Level 2, Containers:** the deployable or runnable parts, like the web app, API, database and queue.
- **Level 3, Components:** the main modules inside one container.
- **Level 4, Code:** classes, rarely drawn.

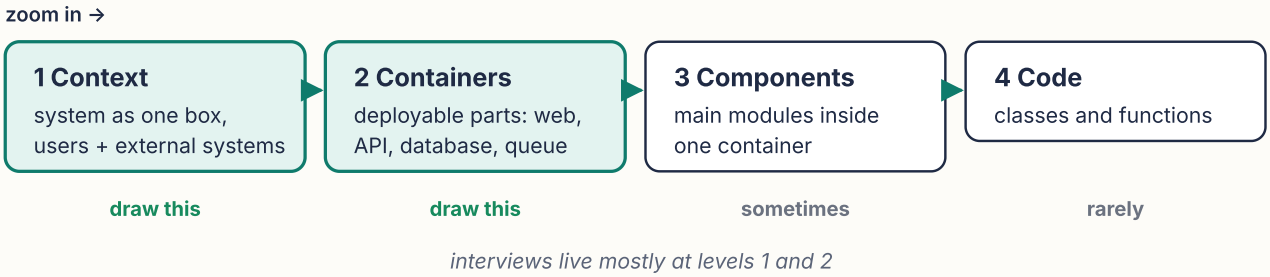



**Figure 4.** C4 levels. A "container" in C4 means anything that runs or stores data on its own, not specifically a Docker container.

**COMMON MISTAKES**

- Unlabelled arrows: nobody knows whether it's a call, a message or a replication stream.
- Drawing a queue as a line instead of a box, so its backlog and consumers are invisible.
- One giant diagram mixing reads, writes, background jobs and admin flows.
- Jumping to class-level detail (C4 level 4) before the containers are agreed.
- Arrows that cross the whole board; re-lay the boxes so flows move left to right.

## 6 Key terms

**Synchronous**: caller waits for the response

**Asynchronous**: sender hands off a message and moves on

**Read path / write path**: the components a read or a write touches

**Fan-out on write**: copy a new item into each follower's feed at write time

**Sequence diagram**: participants as columns, time flowing down

**C4 model**: context, containers, components, code

## 7 Interview questions

**Q1. How do you show sync vs async flows on a diagram?** BASIC

Solid arrows for synchronous request-response calls (HTTP, gRPC). Dashed arrows through an explicit queue or topic box for asynchronous messages. Number the steps.

**Q2. What is the C4 model?** BASIC

Simon Brown's four levels of zoom: context (system + users + external systems), containers (deployable parts), components (modules inside one container), code (classes). Interviews use levels 1 and 2.

**Q3. How do you lay out a system diagram?** BASIC

Left to right in layers: clients, edge (DNS, CDN, load balancer or gateway), services, data stores. Label every box with what it is and every arrow with what flows.

**Q4. Draw the write and read paths for a news feed.** SCENARIO

Write: post service stores the post, publishes to a queue, fan-out workers push the post ID into each follower's Redis list. Read: feed service reads the precomputed list, fetches details from a cache.

**Q5. When would you add a sequence diagram?** SCENARIO

For one critical request with several hops, like checkout or login, where order, timeouts and retries matter. Example: payment times out at 2 s, is retried with the same idempotency key.

**Q6. Why draw the queue as its own box?** SCENARIO

It is a real component: it can fill up, needs consumers, and has its own scaling and failure modes. As a box you can label its throughput, say 5,000 messages/s, and its consumers.

**Q7. Why separate read and write paths if they share a database?** TRICKY

They scale differently. A 100 : 1 read-heavy system adds replicas and caches to the read path while the write path stays on one primary. Separate drawings make that obvious.

**Q8. Is a C4 "container" a Docker container?** TRICKY

No. It is any separately running or data-storing unit: a web app, a mobile app, a database, a queue. It may or may not be packaged in Docker.

Next: SD-D-013 · Identifying Bottlenecks backendroadmap.com

# Identifying Bottlenecks

*find what breaks first, before production does*

Watch the 4-min video

**In one line:** the bottleneck is the resource that limits throughput for the whole system; find it by writing peak load and capacity next to every component on the critical path, and the one with the highest load ÷ capacity ratio is it.

## 1 A system is only as fast as its narrowest point

Your design handles 100,000 requests per second on paper. Then launch day comes and everything slows to a crawl. Not because every part is too small, but because *one* part is.

A **bottleneck** is the resource that limits throughput for the whole system. A request passes through a load balancer, app servers, a cache and a database. If the database can take 5,000 writes per second, the whole write path tops out at 5,000, no matter how many app servers you add. **Scaling anything else just moves the queue.**

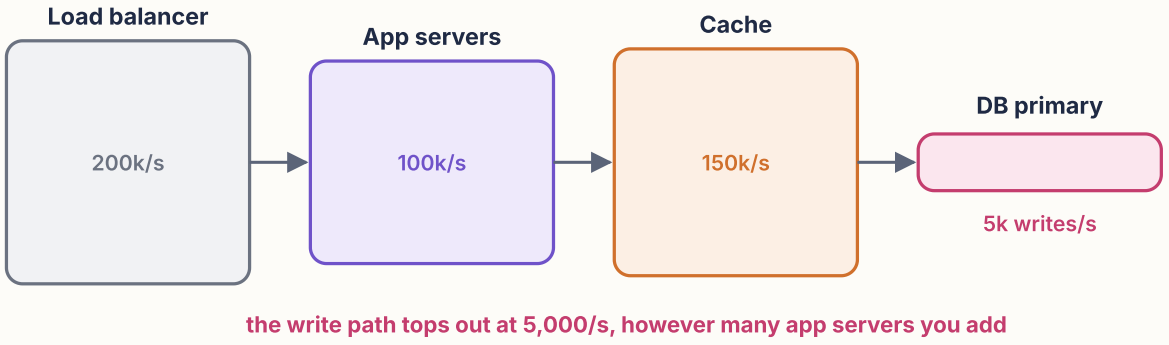



**Figure 1.** Throughput of a path is the minimum capacity along it. Here the single database primary is the narrow pipe.

## 2 Spotting one in a running system

Look for **saturation**: CPU pinned near 100%, disks maxed out, connection pools exhausted, or a queue whose depth keeps growing. Latency climbs sharply as a resource nears saturation, because requests start waiting in line. The resource that saturates first is the one to fix.

The **USE method** (from Brendan Gregg) makes this systematic: for every resource, check **U**tilization, **S**aturation and **E**rrors.

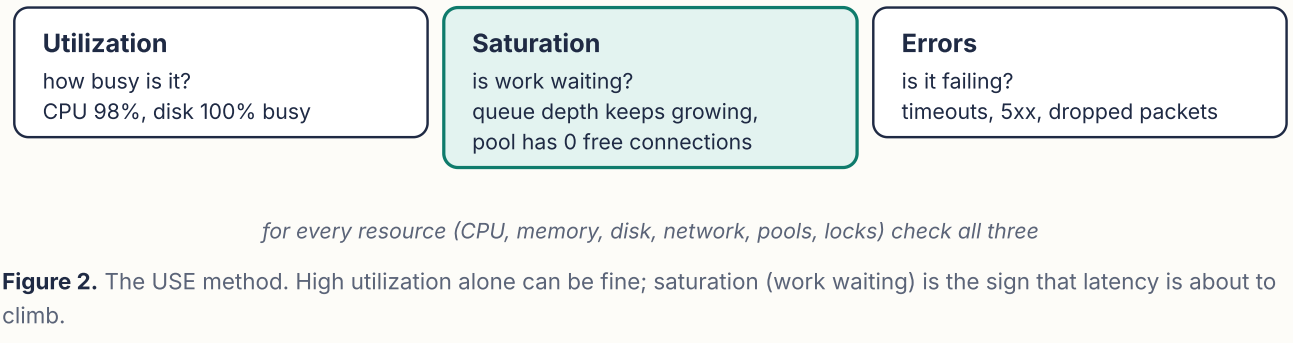



**Figure 2.** The USE method. High utilization alone can be fine; saturation (work waiting) is the sign that latency is about to climb.

## 3 The usual suspects in a design

| Suspect | Why it limits | Typical fix |
|---|---|---|
| **Database writes** | A single primary takes every write. | Batch, buffer, shard |
| **Hot partition** | One key, like a celebrity account, gets far more traffic than the rest. | Split or salt the key, cache it |
| **Network and egress** | A link or NIC fills up, often with large payloads. | CDN, compression, smaller responses |
| **Single consumers, global locks** | One worker drains a queue, or one lock serializes everything. | Partition the work, remove the lock |

## 4 Finding it in an interview: load ÷ capacity

Use your estimates. Walk the **critical path** of each major request. Next to every component, write the **peak load** it receives and the **capacity** it has. Divide one by the other. **The component with the highest ratio is your bottleneck.**

Remember that load concentrates: reads multiply through fan-out (one feed request may read 50 items), and peaks are several times the average.

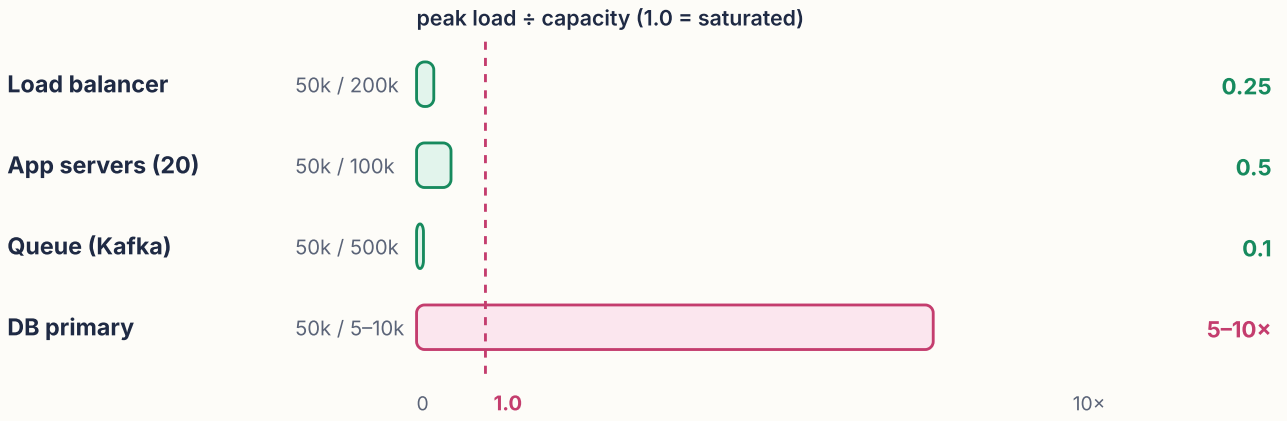



**Figure 3.** The workbook case at 50,000 writes/s peak. Capacities other than the database are illustrative. Only the primary is above 1.0, by 5 to 10 times.

**WORKED EXAMPLE: 50,000 WRITES PER SECOND**

Your estimates say **50,000 writes/s at peak**. A single relational primary on good hardware handles maybe **5,000–10,000** simple writes per second. So the write path is the bottleneck, by **5 to 10 times**.

The fixes:

- **Batch writes:** one insert of 500 rows costs far less than 500 inserts.
- **Buffer through a queue:** absorb the peak and write at a steady rate the database can take.
- **Shard** the database across about **10 primaries** (50,000 ÷ 5,000).
- **Move to a write-optimized store** like Cassandra, built for high write throughput.

## 5 Fixing one reveals the next

After sharding, maybe the queue consumers fall behind, or one shard runs hot. That is normal. Address bottlenecks **in order of impact**, and confirm them with **load tests and metrics**, not guesses. Say this out

loud in interviews: it shows you think about the system as a whole.

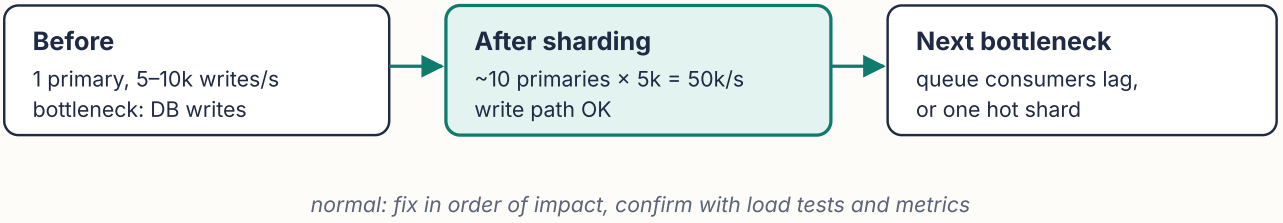



**Figure 4.** Bottlenecks move. Plan for the next one as soon as you fix the current one.

**COMMON MISTAKES**

- Adding app servers when the database is the limit: it just grows the queue in front of the database.
- Using average load instead of peak, and forgetting fan-out.
- Ignoring hot keys: total capacity is fine but one partition melts.
- Guessing the bottleneck instead of measuring it (USE, load tests).
- Stopping after one fix, without asking what breaks next.

## 6 Key terms

**Bottleneck**: the resource that limits whole-system throughput

**Saturation**: more work than capacity, so work waits

**Hot partition**: one key or shard taking far more load than others

**Critical path**: the components a request must pass through

**USE method**: utilization, saturation, errors per resource

**Fan-out**: one request turning into many downstream calls

## 7 Interview questions

**Q1. What is a bottleneck?** BASIC

The resource that limits throughput for the whole system. A path's throughput is the minimum capacity along it: a 5,000 writes/s primary caps the write path at 5,000 however many app servers you add.

**Q2. How do you find the bottleneck in your design?** BASIC

Walk the critical path of each request, write peak load and capacity next to every component, and divide. The highest ratio, the one closest to saturation, is the bottleneck.

**Q3. What signals a bottleneck in production?** BASIC

Saturation: CPU near 100%, disks maxed, an exhausted connection pool, or a queue whose depth keeps growing, with latency climbing sharply. Check each resource with USE.

**Q4. Peak is 50,000 writes/s and one primary does 5–10k. What do you do?** SCENARIO

It is 5–10× over. Batch and buffer writes through a queue, shard across about 10 primaries, or move to a write-optimized store like Cassandra. Then look for the next bottleneck.

**Q5. Total traffic is fine but one shard is at 100%. Why, and what's the fix?** SCENARIO

A hot partition: one key, like a celebrity account, gets far more traffic. Cache that key, split it across sub-keys (salting), or give it dedicated capacity.

**Q6. After sharding, the queue's consumer lag keeps growing. What happened?** SCENARIO

The bottleneck moved. Consumers now can't keep up. Add consumers and partitions (one consumer per partition in Kafka), or batch their writes.

**Q7. A server is at 95% CPU but latency is fine. Is it the bottleneck?** TRICKY

Not yet: high utilization without saturation can be healthy. But it has no headroom; a small spike will start queueing and latency will jump. Watch run-queue length, not just CPU %.

**Q8. A feed request fetches 50 posts. Peak is 20,000 feed loads/s. What load does the post store see?** TRICKY

Fan-out multiplies it: 20,000 × 50 = 1 million item reads/s, unless batched or cached. Fan-out is why the obvious tier is often not the bottleneck.

Next: SD-D-014 · Identifying SPOFs backendroadmap.com

# Identifying SPOFs

*ask "what if this dies?" for every box and arrow*

Watch the 3-min video

**In one line:** walk every box, arrow and external dependency and ask "what if this dies?"; anything whose loss stops the system is a SPOF, removed with redundancy across zones, automatic failover, graceful degradation, and timeouts with circuit breakers.

## 1 Redundant servers, still down

Your app runs on 20 servers across three zones. Fully redundant? Then your login provider goes down and nobody can sign in. Not one of your servers failed. A single point of failure was hiding in a dependency nobody drew on the diagram.

A **single point of failure (SPOF)** is any component whose failure takes down the whole system, *or a critical feature*. It isn't only servers. It can be a database primary, a load balancer, a network link, a config store, an expiring certificate, one region, or one person who knows how to deploy.

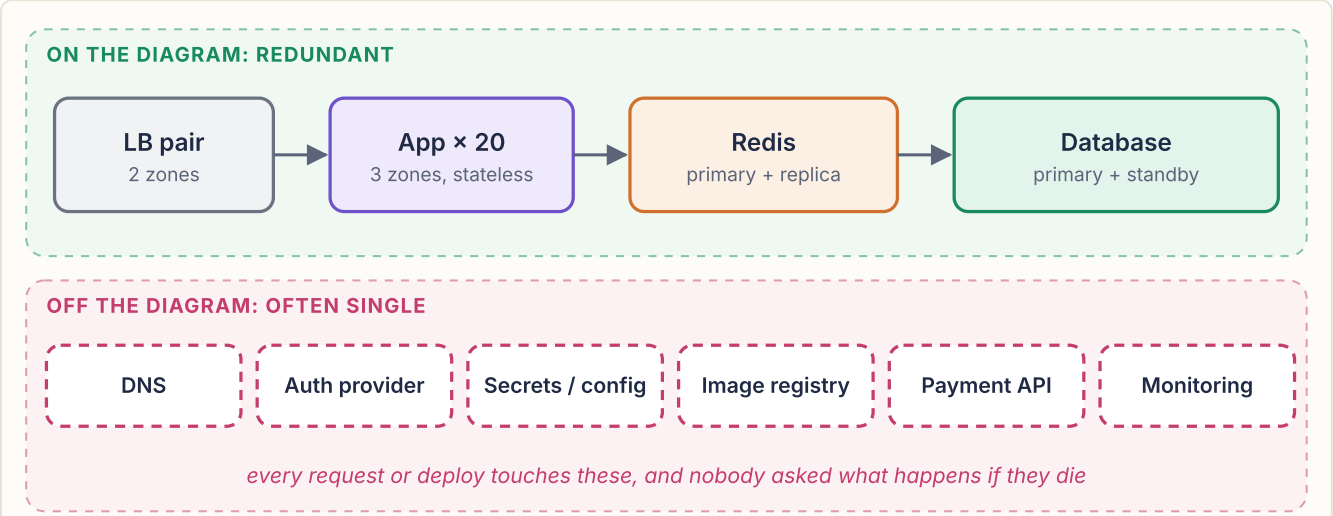



**Figure 1.** The drawn system is redundant. The dependencies around it, each used by every request or every deploy, are where outages hide.

## 2 The method: box by box, arrow by arrow

Walk your diagram one box at a time and ask:

1. **What happens if this dies?**
2. Is there another copy that takes over?
3. How quickly, and is it automatic or by hand?

If the answer is "the whole system stops", you have found a SPOF. Do the same for **every arrow**, because links fail too: a single VPN tunnel, one cross-region link, one NAT gateway.

## 3 The dangerous ones are off the diagram

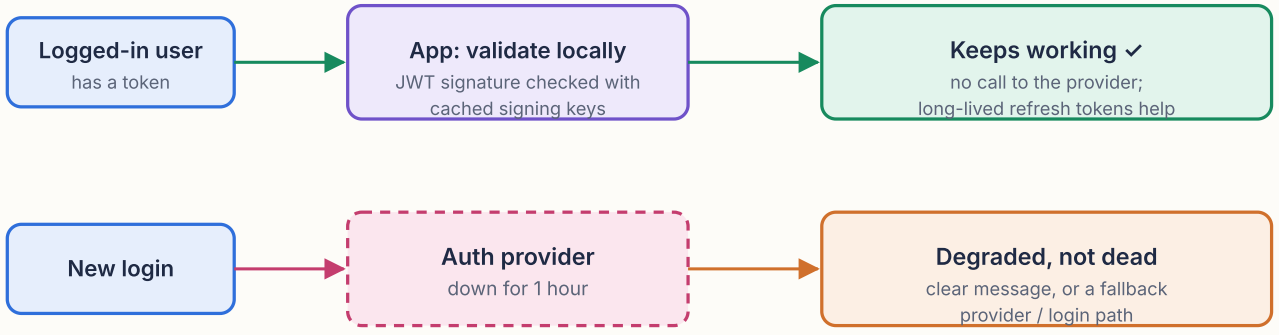
DNS, your identity or auth provider, secrets and config stores, the container registry, a third-party payment API, even your monitoring. Build a **dependency map**: list every external system the critical path

touches, and treat each one as a box you must question.

**WORKED EXAMPLE: THE AUTH PROVIDER GOES DOWN**

Every login calls an external auth provider. It goes down for an hour and nobody can log in, even though all your servers are healthy. Mitigations:

- **Keep logged-in users working:** validate tokens locally with **cached signing keys**, so each request does not call the provider.
- **Longer-lived refresh tokens**, so sessions survive a short outage.
- A **clear degraded message** for new logins instead of a timeout.
- For critical systems, a **second provider** or a fallback login path.




*your 20 servers are healthy in both rows; only the dependency failed*

**Figure 2.** Local token validation turns an auth outage from "nobody can use the app" into "only new logins are affected".

## 4 The general fixes




**Figure 3.** The same four tools remove or soften almost every SPOF.

Add **redundancy**: run at least two of everything, across zones. Add **automatic failover** with health checks, so recovery doesn't wait for a human. **Degrade gracefully**: if recommendations fail, still show the page. And add **timeouts and circuit breakers**, so a dead dependency fails fast instead of hanging every request and exhausting your thread pools.

## 5 Walk it out loud in an interview

| Component | If it dies... | How it survives |
|---|---|---|
| **Load balancer** | No traffic gets in | A redundant pair, or a managed load balancer across zones |
| **App servers** | Capacity drops | Stateless, many instances; others absorb the load |
| **Cache** | Every read hits the database | Replicas, and a database sized to survive a cold cache |
| **Database** | No writes, maybe no reads | Primary with a standby and automatic failover |
| **DNS, auth** | Nobody finds you, or nobody logs in | Second DNS provider; local token validation, fallback login |

You can **accept a SPOF** when removing it isn't worth the cost, say a single-region internal tool, but say so explicitly and name the risk.

COMMON MISTAKES

- Only checking servers, never arrows or external dependencies.
- A standby with manual failover at 3 a.m.: the outage lasts as long as it takes to wake someone.
- A cache that is redundant, but a database that collapses when the cache is cold.
- No timeouts: one hung dependency ties up every thread and takes healthy services down with it.
- Silently accepting a SPOF instead of naming it and its cost.

## 6 Key terms

**SPOF**: a component whose loss stops the system or a critical feature

**Automatic failover**: health checks promote a backup without a human

**Circuit breaker**: stops calling a failing dependency for a while

**Dependency map**: every external system the critical path touches

**Graceful degradation**: keep core features when extras fail

**Cold cache**: an empty cache after a restart or failover

## 7 Interview questions

**Q1. How do you find single points of failure?** BASIC

Walk every component and arrow and ask what happens if it dies, is there a copy, and how fast it takes over. Include hidden dependencies: DNS, auth, secrets and config stores, registries, third-party APIs.

**Q2. How do you remove them?** BASIC

Redundancy across zones (at least two of everything), automatic failover with health checks, graceful degradation, and timeouts with circuit breakers.

**Q3. Is a SPOF always a server?** BASIC

No. It can be a network link, a config store, an expiring certificate, one region, or one person who knows how to deploy.

**Q4. The external auth provider is down for an hour. What still works?** SCENARIO

If tokens are validated locally with cached signing keys, logged-in users keep working; refresh tokens extend sessions. New logins get a clear degraded message, or a second provider.

**Q5. Your Redis cluster fails over and the cache is empty. What happens?** SCENARIO

Every read hits the database at once. If it was sized for a 90% hit rate, it suddenly sees 10× the reads. Size the database for a cold cache, warm the cache, or rate-limit until it refills.

**Q6. A recommendations service hangs. Why does the whole page time out?** SCENARIO

No timeout: each request waits, threads pile up and the pool runs dry. Add a timeout (say 200 ms), a circuit breaker, and render the page without recommendations.

**Q7. Your monitoring runs in the same region as the app. Problem?** TRICKY

Yes. A regional outage blinds you exactly when you need it, and alerts may never fire. Run alerting or external probes from outside the failure domain.

**Q8. When is it fine to keep a SPOF?** TRICKY

When removing it costs more than the outage risk, like a second auth provider for an internal tool. Say so explicitly, estimate the impact, and have a recovery plan.

Next: SD-G-001 · Load Balancer Fundamentals backendroadmap.com